

Common DP Patterns — Part 1

A complete placement + competitive programming guide to 14 essential dynamic programming patterns.

1. FIBONACCI STYLE

1. Overview

Fibonacci-style DP is the simplest form of dynamic programming. It involves problems where the current state depends **directly on one or two previous states**, forming a linear recurrence relation. The classic Fibonacci sequence is $F(n) = F(n-1) + F(n-2)$ with base cases $F(0) = 0$, $F(1) = 1$. Any problem that follows a similar additive recurrence on a linear chain belongs here.

2. Intuition

Simple explanation: To compute the n th term, you only need the previous two (or a few) terms. Instead of recomputing them recursively (which explodes exponentially), you store them in variables or an array and build up iteratively.

Analogy: Imagine climbing a staircase where you can only see the step you're on and the one behind you. You don't need to remember every step you've ever taken — just the last two. That's the Fibonacci insight: the optimal substructure is a simple sliding window of history.

Step-by-step reasoning:

1. Define the recurrence: $dp[i] = dp[i-1] + dp[i-2]$ (or similar linear combination).
2. Identify base cases: typically $dp[0]$, $dp[1]$.
3. Compute iteratively from 2 to n , keeping only the last few values.
4. Return $dp[n]$.

Why it works: The problem satisfies **optimal substructure** (the answer for n is built from answers for $n-1$ and $n-2$) and **overlapping subproblems** (the same subproblems — e.g., $F(3)$ — are needed many times). DP eliminates redundant recursion.

3. When to Use It

- Problems that ask for the **n th term** of a sequence defined by a recurrence.
- Problems where each step depends on **one or two previous steps**.
- Counting problems on a linear chain (e.g., number of ways to reach step n).
- "How many ways..." with fixed transition options (1-step, 2-step, etc.).
- Problems with **linear DP** where the state is a single integer index.

Trigger phrases: "nth term", "how many ways", "tiling", "Fibonacci-like", "linear recurrence", "first n ", "count the number of sequences".

4. When Not to Use It

- When the recurrence involves **more than a few previous states** (say, > 3 or 4) — a full DP array is still fine, but the "sliding window" memory optimization becomes unwieldy.
- When the problem has **multiple dimensions** or complex state beyond a single index — use grid DP or state DP instead.

- When the problem asks for **min/max with constraints** that don't decompose linearly — could be greedy, or need more complex DP.
- When n is astronomically large ($10^{18}+$) — use matrix exponentiation, not iterative DP.
- When the recurrence is not additive but involves multiplication, modulo, or non-linear operations that break the simple pattern.

5. Core Concepts

Concept	Explanation	Why It Matters
Recurrence relation	A formula expressing $dp[n]$ in terms of smaller $dp[k]$	The heart of Fibonacci-style DP; everything follows from this
Base cases	Smallest known values (e.g., $dp[0]$, $dp[1]$)	Without them, recurrence has no foundation
Memoization vs Tabulation	Top-down (recursive + cache) vs bottom-up (iterative)	Tabulation is usually faster and avoids stack overflow
Space optimization	Keeping only last k values instead of full array	Reduces memory from $O(n)$ to $O(k)$
Modular arithmetic	Computing $dp[i] \% MOD$ when numbers get large	Required for most CP and placement problems

6. Step-by-Step Algorithm

Tabulation (Bottom-Up):

1. If $n \leq 1$, return n directly (handle base cases).
2. Create array dp of size $n+1$.
3. Set $dp[0] = 0$, $dp[1] = 1$.
4. For $i = 2$ to n :
 - $dp[i] = dp[i-1] + dp[i-2]$.
5. Return $dp[n]$.

Space-Optimized:

1. If $n \leq 1$, return n .
2. Set $prev2 = 0$, $prev1 = 1$.
3. For $i = 2$ to n :
 - $curr = prev1 + prev2$.
 - $prev2 = prev1$.
 - $prev1 = curr$.
4. Return $prev1$.

7. Dry Run

Input: $n = 6$

i	$dp[i-2]$ (prev2)	$dp[i-1]$ (prev1)	$dp[i]$ (curr)
0	-	-	0 (base)
1	-	-	1 (base)

i	dp[i-2] (prev2)	dp[i-1] (prev1)	dp[i] (curr)
2	0	1	1
3	1	1	2
4	1	2	3
5	2	3	5
6	3	5	8

Answer: $F(6) = 8$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Fibonacci: Space-Optimized -----
// Time: O(n), Space: O(1)
int fibonacci(int n) {
    if (n <= 1) return n;

    int prev2 = 0; // F(0)
    int prev1 = 1; // F(1)
    int curr;

    for (int i = 2; i <= n; ++i) {
        curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

// ----- Fibonacci: With array (for reference) -----
int fibonacciArray(int n) {
    if (n <= 1) return n;
    vector<int> dp(n + 1);
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}

// ----- Fibonacci: Modulo (common in CP) -----
const int MOD = 1e9 + 7;
int fibonacciMod(int n) {
    if (n <= 1) return n;
    int prev2 = 0, prev1 = 1, curr;
    for (int i = 2; i <= n; ++i) {
        curr = (prev1 + prev2) % MOD;
        prev2 = prev1;
        prev1 = curr;
    }
}
```

```

    return prev1;
}

// ----- Fibonacci: Top-down with memoization -----
int fibMemo(int n, vector<int>& memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

int fibonacciTopDown(int n) {
    vector<int> memo(n + 1, -1);
    return fibMemo(n, memo);
}

// ----- Example usage -----
int main() {
    int n = 10;
    cout << "F(" << n << ") = " << fibonacci(n) << "\n";           // 55
    cout << "F(" << n << ") mod = " << fibonacciMod(n) << "\n";       // 55
    return 0;
}

```

9. Python Implementation

```

# ----- Fibonacci: Space-Optimized -----
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for i in range(2, n + 1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr
    return prev1

# ----- Fibonacci: With array -----
def fibonacci_array(n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

# ----- Fibonacci: Modulo -----
MOD = 10**9 + 7
def fibonacci_mod(n: int) -> int:
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for i in range(2, n + 1):
        curr = (prev1 + prev2) % MOD
        prev2, prev1 = prev1, curr
    return prev1

```

```
# ----- Fibonacci: Top-down -----
def fibonacci_top_down(n: int) -> int:
    memo = [-1] * (n + 1)
    def dfs(k: int) -> int:
        if k <= 1:
            return k
        if memo[k] != -1:
            return memo[k]
        memo[k] = dfs(k - 1) + dfs(k - 2)
        return memo[k]
    return dfs(n)

# Example
print(fibonacci(10))    # 55
```

10. Code Explanation

- **Base case handling:** `if (n <= 1) return n;` — catches `n = 0` and `n = 1` immediately.
- **Sliding variables:** `prev2` and `prev1` store `F(i-2)` and `F(i-1)` respectively. At each step, `curr = prev1 + prev2` computes `F(i)`.
- **Update step:** `prev2 = prev1; prev1 = curr;` shifts the window forward by one position.
- **Modulo version:** `(prev1 + prev2) % MOD` keeps numbers within bounds. Essential when `n` is large (e.g., `F(10^5)` has ~20k digits).
- **Top-down version:** Uses recursion with memoization. The `memo` array stores computed values. `fibMemo(n-1, memo) + fibMemo(n-2, memo)` builds the answer recursively. Though elegant, it uses $O(n)$ stack space and is slightly slower due to function call overhead.
- **Why space optimization works:** `F(i)` never needs values older than `F(i-2)`, so the sliding window of size 2 is sufficient.

11. Complexity Analysis

Variant	Time	Space	Notes
Naïve recursion	$O(2^n)$	$O(n)$	Exponential — never use this
Top-down memoization	$O(n)$	$O(n)$	Recursion stack + array
Bottom-up array	$O(n)$	$O(n)$	Simple, good for understanding
Space-optimized	$O(n)$	$O(1)$	Best for most placement/CP problems
Matrix exponentiation	$O(\log n)$	$O(1)$	For $n > 10^7$ (rare in placements)

12. Common Patterns

Pattern	Identification	Approach	Example Problems
Simple nth term	"Find the nth Fibonacci number"	Direct recurrence	Fibonacci (LeetCode 509)
Counting ways	"How many ways to climb / arrange / tile"	$dp[i] = dp[i-1] + dp[i-2]$	Climbing Stairs (LeetCode 70)
Tiling problems	"Tile a $2 \times n$ board with 2×1 tiles"	$dp[i] = dp[i-1] + dp[i-2]$	Tiling Dominoes (GFG)

Pattern	Identification	Approach	Example Problems
N-th tribonacci	Depends on last 3 terms	$dp[i] = dp[i-1] + dp[i-2] + dp[i-3]$	Tribonacci (LeetCode 1137)
Counting with constraints	"No two 1s adjacent"	$dp[i][0/1] = \text{two-state Fibonacci}$	Count Binary Strings (GFG)

13. Common Mistakes

- **Forgetting base cases:** $n = 0$ and $n = 1$ often crash the loop or return wrong values.
- **Off-by-one in array size:** Using $dp[n]$ when array is size n instead of $n+1$.
- **Integer overflow:** $F(50)$ exceeds 32-bit int. Use `long long` (or `uint64_t`) and modulo in CP.
- **Bad recursion without memoization:** Naive recursive Fibonacci is $O(2^n)$ and will TLE/hang for $n > 40$.
- **Incorrect space optimization initialization:** `prev2` and `prev1` must start as $F(0)$ and $F(1)$ — swapping them gives wrong answer.
- **Modulo applied too late:** If addition overflows before applying mod, the result is garbage. Apply mod during addition.

14. Edge Cases

Case	Input	Expected	Why It Matters
$n = 0$	0	0	Base case — return early
$n = 1$	1	1	Base case — return early
$n = 2$	2	1	First computed value
$n = \text{large}$	50	12586269025	Overflows 32-bit — need 64-bit
$n = \text{very large}$	10^6	(mod value)	Must use $O(1)$ space + modulo
$n = \text{negative}$	-5	undefined/error	Guard against invalid input

15. Variations

Variation	What Changes	When Used	Importance
Tribonacci	$dp[i] = dp[i-1] + dp[i-2] + dp[i-3]$	Problems needing 3-term recurrence	Medium — LeetCode 1137
K-step Fibonacci	$dp[i] = \text{sum}(dp[i-1] \dots dp[i-k])$	Generalised counting	Low for placements
Matrix exponentiation	Uses $[[1,1],[1,0]]^n$	When n is enormous (10^{18})	Medium — mostly CP
Fast doubling	Uses identities to compute $F(n)$ in $O(\log n)$	CP contests with large n	Low — niche
DP with constraints	$dp[i][0/1]$ adding state for "last chosen"	Counting with adjacency constraints	High — common in placements

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Matrix Exponentiation	Generalises linear recurrences to $O(\log n)$	Use when $n > 10^7$
Kadane's Algorithm	Also a linear scan DP (max subarray)	Different recurrence: max vs sum
Sliding Window	Also maintains recent values	Window is for subarray properties, not recurrence
Generating Functions	Closed-form (Binet's formula)	Theoretical interest only in CP

17. Practice Problems

Easy

1. **Fibonacci Number** — LeetCode 509 — Direct recurrence — Easy
2. **N-th Tribonacci Number** — LeetCode 1137 — Three-term recurrence — Easy

Medium

1. **Count Ways to Build Good Strings** — LeetCode 2466 — Fibonacci-like with constraints — Medium
2. **Count Number of Ways to Place Houses** — LeetCode 2320 — Fibonacci with adjacency constraint — Medium

Hard

1. **Number of Ways to Rearrange Sticks With K Sticks Visible** — LeetCode 1866 — Advanced linear DP — Hard
2. **Strange Printer II** — LeetCode 1591 — Harder recurrence with intervals — Hard

18. Interview Explanation

"Fibonacci-style DP is the most basic dynamic programming pattern. The idea is that the answer for n depends only on the answer for $n-1$ and $n-2$ — a simple linear recurrence. Instead of recomputing subproblems recursively (which is exponential), we compute iteratively from the base cases upward, keeping only the last two values for $O(1)$ space. It works because of optimal substructure — the problem breaks down into smaller independent subproblems — and overlapping subproblems, since the same small values are reused many times. The key in an interview is to recognise the recurrence, identify base cases, and then optimise space from $O(n)$ to $O(1)$. Common variations include generalising to k previous terms or adding a state dimension for constraints."

19. Revision Notes

- **Key idea:** `dp[i] = sum of last k dp values` (usually $k = 2$).
- **Formula:** $F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$.
- **Template:** `prev2 = base1, prev1 = base2; for i in range(2, n+1): curr = f(prev1, prev2); prev2 = prev1; prev1 = curr;`
- **Complexity:** $O(n)$ time, $O(1)$ space.
- **Traps:** Integer overflow, forgetting modulo, off-by-one in n , base case $n = 0$.
- **Space optimisation:** Only keep last k values (usually 2).

20. Final Cheat Sheet

FIBONACCI STYLE DP

When to use: n th term, linear recurrence, counting ways on a chain

```

Recurrence:  dp[i] = dp[i-1] + dp[i-2] (or similar)
Base cases:  dp[0], dp[1] (problem-specific)
Time:        O(n)
Space:       O(1) with sliding window, O(n) with array

```

Key code:

```

for (int i = 2; i <= n; ++i) {
    curr = prev1 + prev2;
    prev2 = prev1;
    prev1 = curr;
}

```

Edge cases: n = 0, n = 1, overflow, modulo

Variations: Tribonacci, k-step, matrix exponentiation

2. CLIMBING STAIRS

1. Overview

The Climbing Stairs problem is defined as: "You are climbing a staircase with n steps. Each time you can climb 1 or 2 steps. In how many distinct ways can you reach the top?" It is mathematically equivalent to the Fibonacci sequence but shifted: $ways(n) = ways(n-1) + ways(n-2)$, with $ways(1) = 1$, $ways(2) = 2$. This is one of the most common interview DP problems and the gateway to understanding DP as "counting ways" on a state graph.

2. Intuition

Simple explanation: To reach step i , you must have come from either step $i-1$ (by taking 1 step) or step $i-2$ (by taking 2 steps). Therefore, the total ways to reach i is the sum of ways to reach $i-1$ and ways to reach $i-2$. This creates a natural Fibonacci-like recurrence.

Analogy: Think of a vending machine that dispenses a snack. To get snack $\#i$, you must have inserted either 1 coin or 2 coins just before. The number of coin sequences that get you snack $\#i$ is the sum of sequences for snack $\#(i-1)$ and snack $\#(i-2)$.

Step-by-step reasoning:

1. If there is 1 step, there is exactly 1 way: take 1 step.
2. If there are 2 steps, there are 2 ways: (1+1) or (2).
3. For step 3: from step 2 (2 ways) take 1 step, OR from step 1 (1 way) take 2 steps → total 3 ways.
4. Each step i depends only on $i-1$ and $i-2$.

Why it works: The problem has **optimal substructure** (the ways to reach i is built from ways to reach smaller steps) and **overlapping subproblems** (ways(3) is used in computing ways(4) and ways(5)). The choices are independent — the path before $i-1$ doesn't affect the path from $i-1$ to i .

3. When to Use It

- Exactly the Climbing Stairs problem on LeetCode or similar.
- Any problem asking "number of ways to reach the end" with fixed step sizes (1, 2).
- Problems where transitions are simple additive jumps.
- As a warm-up to explain DP in interviews.

Trigger phrases: "Climbing stairs", "number of ways to reach", "can take 1 or 2 steps", "how many distinct ways", "reach the top".

4. When Not to Use It

- When step sizes are **variable and many** (e.g., you can take any step from a set $\{a, b, c, \dots\}$) — still DP, but the state transition becomes a sum over the set, and you may need $dp[i] = \sum(dp[i - \text{step}] \text{ for step in steps})$.
- When steps have **costs/weights** — becomes min-cost climbing stairs, a different DP (minimisation).
- When there are **constraints like "cannot take two 2-steps in a row"** — needs a 2D state (last step taken).
- When n is enormous ($10^{12}+$) and steps are fixed — use matrix exponentiation.
- When there are **blocked steps** — still DP but with conditional transitions.

5. Core Concepts

Concept	Explanation	Why It Matters
State definition	$dp[i]$ = number of ways to reach step i	The core modelling decision
Transition	$dp[i] = dp[i-1] + dp[i-2]$	Captures the two choices
Base cases	$dp[1] = 1, dp[2] = 2$ (or $dp[0] = 1, dp[1] = 1$ for Fibonacci mapping)	Foundation of recurrence
Choice independence	Paths to $i-1$ and $i-2$ are independent	Why we can sum them
Space optimisation	Only need last 2 values	Same as Fibonacci — $O(1)$ space

6. Step-by-Step Algorithm

1. If $n == 1$, return 1.
2. If $n == 2$, return 2.
3. Set $prev2 = 1$ (ways to reach step 1).
4. Set $prev1 = 2$ (ways to reach step 2).
5. For $i = 3$ to n :
 - $curr = prev1 + prev2$.
 - $prev2 = prev1$.
 - $prev1 = curr$.
6. Return $prev1$.

7. Dry Run

Input: $n = 5$

i	ways to i (prev2)	ways to $i-1$ (prev1)	ways to i (curr)
1	-	-	1 (base)
2	-	-	2 (base)
3	1	2	3
4	2	3	5

i	ways to i (prev2)	ways to i-1 (prev1)	ways to i (curr)
5	3	5	8

Answer: 8 ways.

Explanation of ways for n = 5:

1. 1+1+1+1+1
2. 1+1+1+2
3. 1+1+2+1
4. 1+2+1+1
5. 2+1+1+1
6. 1+2+2
7. 2+1+2
8. 2+2+1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Climbing Stairs: Space Optimized -----
// LeetCode 70
int climbStairs(int n) {
    if (n <= 2) return n;

    int prev2 = 1; // ways to reach step 1
    int prev1 = 2; // ways to reach step 2
    int curr;

    for (int i = 3; i <= n; ++i) {
        curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

// ----- Climbing Stairs: With array -----
int climbStairsArray(int n) {
    if (n <= 2) return n;
    vector<int> dp(n + 1);
    dp[1] = 1;
    dp[2] = 2;
    for (int i = 3; i <= n; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}

// ----- Climbing Stairs: With modulo (large n) -----
const int MOD = 1e9 + 7;
int climbStairsMod(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2, curr;
```

```

    for (int i = 3; i <= n; ++i) {
        curr = (prev1 + prev2) % MOD;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

// ----- Example usage -----
int main() {
    int n = 5;
    cout << "Ways to climb " << n << " stairs: " << climbStairs(n) << "\n"; // 8
    return 0;
}

```

9. Python Implementation

```

# ----- Climbing Stairs: Space Optimized -----
def climb_stairs(n: int) -> int:
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for i in range(3, n + 1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr
    return prev1

# ----- Climbing Stairs: With array -----
def climb_stairs_array(n: int) -> int:
    if n <= 2:
        return n
    dp = [0] * (n + 1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

# ----- Climbing Stairs: Modulo -----
MOD = 10**9 + 7
def climb_stairs_mod(n: int) -> int:
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for i in range(3, n + 1):
        curr = (prev1 + prev2) % MOD
        prev2, prev1 = prev1, curr
    return prev1

# Example
print(climb_stairs(5)) # 8

```

10. Code Explanation

- **Base case:** $n \leq 2$ returns n directly — closes the trivial cases.

- **Recurrence:** $\text{curr} = \text{prev1} + \text{prev2}$ is the Fibonacci recurrence shifted (1, 2 instead of 0, 1).
- **Space optimisation:** Only two variables needed since $\text{dp}[i]$ only depends on $\text{dp}[i-1]$ and $\text{dp}[i-2]$. This is identical to the Fibonacci pattern.
- **Array version:** Useful for understanding but wasteful for this problem. Some problems might need the full array for backtracking (e.g., reconstructing one valid path), but Climbing Stairs usually doesn't.
- **Modulo version:** Essential for CP when n can be up to 10^5 or 10^6 and the answer must fit in 32/64-bit output.

11. Complexity Analysis

Variant	Time	Space	Notes
Naive recursion	$O(2^n)$	$O(n)$	Exponential — never use
Top-down memo	$O(n)$	$O(n)$	Acceptable but suboptimal space
Bottom-up array	$O(n)$	$O(n)$	Fine for interviews
Space-optimised	$O(n)$	$O(1)$	Best for all practical purposes
Matrix exponentiation	$O(\log n)$	$O(1)$	For astronomically large n

12. Common Patterns

Pattern	Identification	Approach	Example
Standard climb	"Take 1 or 2 steps, count ways"	$\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$	LeetCode 70
Min cost climb	"Each step has cost, minimise total"	$\text{dp}[i] = \text{cost}[i] + \min(\text{dp}[i-1], \text{dp}[i-2])$	LeetCode 746
K-step climb	"Can take 1..k steps"	$\text{dp}[i] = \sum(\text{dp}[i-j] \text{ for } j=1..k)$	Generalized climb
Climb with obstacles	"Some steps are blocked"	$\text{dp}[i] = (\text{blocked} ? 0 : \text{dp}[i-1] + \text{dp}[i-2])$	Variation
Climb with adjacent restriction	"Cannot take 2 steps twice in a row"	Add state for last move	Medium-hard variation

13. Common Mistakes

- **Confusing with Fibonacci indices:** Climbing stairs starting from 1 gives $[1, 2, 3, 5, 8, \dots]$ while Fibonacci starting from 0 gives $[0, 1, 1, 2, 3, 5, 8, \dots]$. $\text{climbStairs}(n) = \text{fibonacci}(n+1)$.
- **Forgetting $n = 0$:** Some definitions say 0 steps $\rightarrow$ 1 way (stay at ground). Be consistent.
- **Wrong base case for $n = 2$:** Some set $\text{dp}[2] = 1$ (thinking only 1+1). The correct answer is 2 (1+1 and 2).
- **Using int for large n :** $n = 46$ overflows 32-bit. Use `long long` or modulo.
- **Over-complicating:** This is literally Fibonacci. Don't introduce unnecessary states.

14. Edge Cases

Input	Expected	Reason
$n = 0$	1 (or 0, depending on definition)	Define clearly before coding
$n = 1$	1	Only 1 step
$n = 2$	2	1+1, 2

Input	Expected	Reason
n = 45	1836311903	Max for signed 32-bit
n = 46	2971215073	Overflows 32-bit int

15. Variations

Variation	Description	Importance
Min Cost Climbing Stairs	Each step has cost, find min cost to reach top	High — LeetCode 746
Triple Step	Can take 1, 2, or 3 steps	Medium — Cracking the Coding Interview
Climbing with variable steps	Array <code>steps[]</code> of allowed jump sizes	Medium
Climbing with adjacency constraint	Cannot take two big steps in a row	Medium-Hard
Climbing on a circular staircase	Can go from top to bottom?	Low — rare

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Fibonacci	Same recurrence, different base	<code>climbStairs(n) == fib(n+1)</code>
Min Cost Climbing Stairs	Min instead of sum, cost array	Different operation, same structure
Coin Change (Ways)	Similar counting on values	Coin change allows any number of each coin
Unique Paths	2D version: grid instead of line	Same recurrence in 2D

17. Practice Problems

Easy

1. **Climbing Stairs** — LeetCode 70 — Direct problem — Easy
2. **Min Cost Climbing Stairs** — LeetCode 746 — Minimisation variant — Easy

Medium

1. **Count Number of Ways to Place Houses** — LeetCode 2320 — Fibonacci with state — Medium
2. **Number of Ways to Divide a Long Corridor** — LeetCode 2147 — Combinatorial DP — Medium

Hard

1. **Number of Ways to Stay in the Same Place After Some Steps** — LeetCode 1269 — 2D climb-like DP — Hard
2. **Strange Printer** — LeetCode 664 — Interval DP, distantly related — Hard

18. Interview Explanation

"Climbing Stairs is the classic introduction to DP. The key insight is that to reach step i , you must have come from either step $i-1$ or step $i-2$. Since those are independent choices, the total ways is the sum. This gives the recurrence $dp[i] = dp[i-1] + dp[i-2]$, which is exactly Fibonacci shifted by one. I handle base cases — 1 way for 1

step, 2 ways for 2 steps — then compute iteratively. I can optimise space to $O(1)$ by keeping only the last two values since the recurrence only depends on them. Time is $O(n)$. If asked about large n , I mention matrix exponentiation for $O(\log n)$."

19. Revision Notes

- **Key idea:** Ways to reach i = ways to reach $i-1$ + ways to reach $i-2$.
- **Formula:** $dp[i] = dp[i-1] + dp[i-2]$, base $dp[1]=1$, $dp[2]=2$.
- **Relation to Fibonacci:** $climbStairs(n) = fib(n+1)$.
- **Template:** Same as Fibonacci but with base 1, 2 instead of 0, 1.
- **Complexity:** $O(n)$ time, $O(1)$ space.
- **Traps:** $n=0$ handling, overflow at $n=46$, confusing with Fibonacci base.

20. Final Cheat Sheet

CLIMBING STAIRS

When to use: Count ways to reach end with fixed step sizes (1, 2)

Recurrence: $dp[i] = dp[i-1] + dp[i-2]$

Base cases: $dp[1] = 1$, $dp[2] = 2$

Time: $O(n)$

Space: $O(1)$

Key code:

```
prev2 = 1, prev1 = 2;
for (i = 3..n) { curr = prev1 + prev2; prev2 = prev1; prev1 = curr; }
return prev1;
```

Edge cases: $n = 0$, $n = 1$, $n = 2$, overflow at $n \geq 46$

Variations: Min cost climb, k-step climb, climb with obstacles

3. HOUSE ROBBER

1. Overview

House Robber is a classic DP problem: You are a professional robber planning to rob houses along a street. Each house has a certain amount of money. The constraint is that you **cannot rob two adjacent houses** (the police will catch you). Find the maximum amount you can rob. The recurrence is $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ — at each house, you decide to either skip it or rob it.

2. Intuition

Simple explanation: At each house, you have two choices:

1. **Skip** this house — the maximum you can have is the same as the max up to the previous house.
2. **Rob** this house — you cannot rob the previous house, so you take the max from two houses ago plus this house's money.

You take the maximum of the two choices.

Analogy: Imagine you're picking fruit from a line of trees, but you can't pick from two adjacent trees. At each tree, you decide: "Do I skip this tree and keep what I've collected so far, or do I pick this tree plus everything I had from two trees ago?"

Step-by-step reasoning:

1. At house 0 (first house), you can only rob it.
2. At house 1, you take max of house 0 and house 1.
3. At house 2, you either skip (keep max from house 1) or rob house 2 + max from house 0.
4. For each subsequent house, the decision is based on the two best previous values.

Why it works: This is a **decision DP** where the state is the index and the value is the maximum loot up to that point. The optimal substructure holds because the decision at i only depends on the optimal solutions to $i-1$ and $i-2$ — no need to know the exact configuration of houses chosen before.

3. When to Use It

- Array of values with **adjacency constraints** (cannot pick two consecutive elements).
- Maximisation problems on a **linear sequence** with a "skip-or-take" pattern.
- Problems where the trade-off is between "take now + skip next" vs "skip now + possibly take next".

Trigger phrases: "Cannot rob adjacent houses", "no two consecutive", "maximum sum without adjacent elements", "thief", "robber", "pick non-adjacent elements".

4. When Not to Use It

- When the constraint is more complex (e.g., "no three consecutive") — needs a larger state ($dp[i][k]$ where k = number of consecutive taken).
- When the array is **circular** (houses in a circle) — still House Robber, but you need two passes (rob first, skip last; skip first, rob last).
- When the array is a **tree** (binary tree with adjacent constraint) — becomes House Robber III (tree DP).
- When there are **no constraints** — just sum all positives (greedy).
- When the constraint is "at most k apart" instead of adjacency — needs modification.

5. Core Concepts

Concept	Explanation	Why It Matters
State	$dp[i]$ = max loot from first i houses	Defines what we compute at each step
Decision	Skip vs rob	Capture the two options
Transition	$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$	Encodes the adjacency constraint
Space optimisation	Only need last 2 dp values	Like Fibonacci but with max
Adjacency constraint	Cannot pick two consecutive elements	The core restriction that makes it DP

6. Step-by-Step Algorithm

1. If $n == 0$, return 0.
2. If $n == 1$, return $\text{nums}[0]$.
3. Set $\text{prev2} = \text{nums}[0]$ (max up to house 0).
4. Set $\text{prev1} = \max(\text{nums}[0], \text{nums}[1])$ (max up to house 1).
5. For $i = 2$ to $n-1$:
 - $\text{curr} = \max(\text{prev1}, \text{prev2} + \text{nums}[i])$.

- `prev2 = prev1.`
 - `prev1 = curr.`
6. Return `prev1`.

7. Dry Run

Input: `nums = [2, 7, 9, 3, 1]`

House i	nums[i]	prev2 (dp[i-2])	prev1 (dp[i-1])	curr (dp[i])	Decision
0	2	-	-	2 (base)	Rob
1	7	-	-	7 (base)	Rob (skip 0)
2	9	2	7	$\max(7, 2+9=11) = 11$	Rob (0 & 2)
3	3	7	11	$\max(11, 7+3=10) = 11$	Skip
4	1	11	11	$\max(11, 11+1=12) = 12$	Rob (0, 2, 4)

Answer: 12 (houses 0, 2, 4 = 2 + 9 + 1 = 12)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- House Robber: Space Optimized -----
// LeetCode 198
int houseRobber(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];

    int prev2 = nums[0]; // dp[i-2]: max up to house 0
    int prev1 = max(nums[0], nums[1]); // dp[i-1]: max up to house 1
    int curr = prev1;

    for (int i = 2; i < n; ++i) {
        curr = max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return curr;
}

// ----- House Robber: Array version -----
int houseRobberArray(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];

    vector<int> dp(n);
    dp[0] = nums[0];
    dp[1] = max(nums[0], nums[1]);

    for (int i = 2; i < n; ++i) {
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i]);
    }
    return dp[n - 1];
}
```

```

    }
    return dp[n - 1];
}

// ----- House Robber II: Circular -----
// LeetCode 213 – houses are arranged in a circle
int robLinear(vector<int>& nums, int start, int end) {
    int prev2 = 0, prev1 = 0;
    for (int i = start; i <= end; ++i) {
        int curr = max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

int houseRobberCircular(vector<int>& nums) {
    int n = nums.size();
    if (n == 1) return nums[0];
    // Case 1: Rob first house, skip last
    int case1 = robLinear(nums, 0, n - 2);
    // Case 2: Skip first house, rob last
    int case2 = robLinear(nums, 1, n - 1);
    return max(case1, case2);
}

// ----- Example usage -----
int main() {
    vector<int> nums = {2, 7, 9, 3, 1};
    cout << "Max loot: " << houseRobber(nums) << "\n"; // 12

    // Circular version
    vector<int> circular = {2, 3, 2};
    cout << "Max loot (circular): " << houseRobberCircular(circular) << "\n"; // 3
    return 0;
}

```

9. Python Implementation

```

# ----- House Robber: Space Optimized -----
def house_robber(nums: list[int]) -> int:
    n = len(nums)
    if n == 0: return 0
    if n == 1: return nums[0]

    prev2 = nums[0]
    prev1 = max(nums[0], nums[1])

    for i in range(2, n):
        curr = max(prev1, prev2 + nums[i])
        prev2, prev1 = prev1, curr
    return prev1

# ----- House Robber: Array version -----
def house_robber_array(nums: list[int]) -> int:
    n = len(nums)

```

```

if n == 0: return 0
if n == 1: return nums[0]

dp = [0] * n
dp[0] = nums[0]
dp[1] = max(nums[0], nums[1])

for i in range(2, n):
    dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])
return dp[n - 1]

# ----- House Robber II: Circular -----
def rob_linear(nums: list[int], start: int, end: int) -> int:
    prev2 = prev1 = 0
    for i in range(start, end + 1):
        curr = max(prev1, prev2 + nums[i])
        prev2, prev1 = prev1, curr
    return prev1

def house_robber_circular(nums: list[int]) -> int:
    n = len(nums)
    if n == 1: return nums[0]
    return max(rob_linear(nums, 0, n - 2), rob_linear(nums, 1, n - 1))

# Example
print(house_robber([2, 7, 9, 3, 1])) # 12
print(house_robber_circular([2, 3, 2])) # 3

```

10. Code Explanation

- **Base cases:** `n == 0` handles empty array (0 loot). `n == 1` returns the only house's value.
- **Initialisation:** `prev2 = nums[0]` (only house 0). `prev1 = max(nums[0], nums[1])` (best of first two).
- **Recurrence:** `curr = max(prev1, prev2 + nums[i])`. The two options are: skip this house (keep `prev1`) or rob this house + what we had two houses ago (`prev2 + nums[i]`).
- **Space optimisation:** Only `prev2` and `prev1` are needed, updated each iteration.
- **Circular version:** Runs two linear passes — one excluding the last house, one excluding the first. The circle constraint means the first and last house are adjacent.
- **Why `max(prev1, prev2 + nums[i])` works:** It implicitly encodes the constraint. If you pick `prev1`, you might have robbed house `i-1`, so you can't rob `i`. If you pick `prev2 + nums[i]`, you skipped `i-1` and are now robbing `i`.

11. Complexity Analysis

Variant	Time	Space	Notes
Basic (space optimised)	$O(n)$	$O(1)$	Best for interview and CP
Array version	$O(n)$	$O(n)$	Useful for backtracking path
Circular (House Robber II)	$O(n)$	$O(1)$	Two linear passes
Tree version (HR III)	$O(n)$	$O(h)$	Tree DP with recursion

12. Common Patterns

Pattern	Identification	Approach	Example
Linear skip-or-take	"Cannot take adjacent"	$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$	LeetCode 198
Circular	"First and last are adjacent"	Two passes: include first/skip last, skip first/include last	LeetCode 213
Tree	"Binary tree, parent+child cannot both be robbed"	DFS with pair return (rob, skip)	LeetCode 337
With at most k apart	"Cannot take elements within distance k"	Sliding window of last k dp values	Variation
With exactly k selections	"Pick k non-adjacent elements"	$dp[i][j] = \max \text{ over } i \text{ with } j \text{ selections}$	Hard variation

13. Common Mistakes

- **Wrong initialisation for $dp[1]$:** Must be $\max(\text{nums}[0], \text{nums}[1])$, not just $\text{nums}[1]$.
- **Forgetting $n == 0$ case:** Many LeetCode problems have empty input.
- **Using $dp[i-2] + \text{nums}[i]$ without checking $i \geq 2$:** Array out of bounds.
- **Confusing with Fibonacci:** House Robber uses $\max$, not $+$.
- **Circular version error:** Not realising you need to check two separate cases.
- **Trying to reconstruct path inefficiently:** Path reconstruction needs the full array.

14. Edge Cases

Input	Expected	Reason
$[]$	0	Empty array
$[5]$	5	Only one house
$[1, 2]$	2	$\max(1, 2) = 2$
$[2, 1, 1, 2]$	4	$2 + 2$ (houses 0 and 3)
$[5, 0, 0, 5]$	10	$5 + 5$ (houses 0 and 3)
$[1, 3, 1, 3, 100]$	103	$100 + 3$
All same $[5, 5, 5, 5]$	10	$5 + 5$ (non-adjacent)

15. Variations

Variation	Description	Importance
House Robber II (Circular)	Houses in a circle — run 2 passes	High — LeetCode 213
House Robber III (Tree)	Binary tree with adjacency constraint	Medium-High — LeetCode 337
Delete and Earn	Choose $\text{nums}[i]$, delete $\text{nums}[i] \pm 1$	Medium — LeetCode 740
Non-adjacent with k selections	Pick exactly k non-adjacent elements	Low — rare in placements

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Fibonacci	Same structure, different operation (+ vs max)	Fibonacci for sum, House Robber for max
Kadane (Max Subarray)	Also a linear DP with max	Kadane requires contiguous elements, House Robber forbids adjacency
Weighted Interval Scheduling	Skip-or-take with weights	More general — intervals instead of fixed positions
0/1 Knapsack	Decision DP with capacities	Knapsack has a weight budget, House Robber has adjacency budget

17. Practice Problems

Easy

1. **House Robber** — LeetCode 198 — Direct implementation — Easy
2. **Maximum Sum of Non-Adjacent Elements** — GFG — Same as House Robber — Easy

Medium

1. **House Robber II** — LeetCode 213 — Circular version — Medium
2. **Delete and Earn** — LeetCode 740 — Transform to House Robber — Medium

Hard

1. **House Robber III** — LeetCode 337 — Tree version — Medium (Hard on some platforms)
2. **Maximum Sum of 3 Non-Overlapping Subarrays** — LeetCode 689 — More complex — Hard

18. Interview Explanation

"House Robber is a classic DP problem where you cannot pick two adjacent elements, and you want the maximum sum. At each house, there are two choices — skip it, keeping the max from the previous house, or rob it, adding its value to the max from two houses ago. The recurrence is $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$. I handle base cases for 0, 1, or 2 houses, then iterate from index 2 onward. Space is $O(1)$ because I only need the last two values. The key insight is that the adjacency constraint turns it into a simple max-based recurrence. For the follow-up with circular houses, I run two linear passes excluding first or last."

19. Revision Notes

- **Key idea:** At each element, $\max(\text{skip}, \text{take})$ where $\text{skip} = dp[i-1]$, $\text{take} = dp[i-2] + \text{nums}[i]$.
- **Formula:** $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$.
- **Base cases:** $dp[0] = \text{nums}[0]$, $dp[1] = \max(\text{nums}[0], \text{nums}[1])$.
- **Template:** `prev2 = nums[0]; prev1 = max(nums[0], nums[1]); for i in 2..n-1: curr = max(prev1, prev2 + nums[i]); prev2 = prev1; prev1 = curr;`
- **Complexity:** $O(n)$ time, $O(1)$ space.
- **Traps:** $n=0$ handling, $dp[1]$ init (must be max), circular = two passes.

20. Final Cheat Sheet

HOUSE ROBBER

When to use: Max sum with no adjacent picks
 Recurrence: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
 Base cases: $dp[0] = \text{nums}[0], dp[1] = \max(\text{nums}[0], \text{nums}[1])$
 Time: $O(n)$
 Space: $O(1)$

Key code:

```
prev2 = nums[0], prev1 = max(nums[0], nums[1]);
for (i = 2..n-1) {
    curr = max(prev1, prev2 + nums[i]);
    prev2 = prev1; prev1 = curr;
}
return prev1;
```

Edge cases: Empty array, single element, circular

Variations: Circular (HR II), Tree (HR III), Delete and Earn

4. MIN/MAX PATH SUM

1. Overview

Minimum (or Maximum) Path Sum problems ask: Given a 2D grid (matrix) of numbers, find a path from the top-left to the bottom-right that minimises (or maximises) the sum of numbers along the path. You can typically move only **right** and **down** in most standard versions, though diagonal moves may be allowed in some variations. The recurrence is $dp[i][j] = \text{grid}[i][j] + \min(dp[i-1][j], dp[i][j-1])$.

2. Intuition

Simple explanation: To reach cell (i, j) , you must come from either the cell above $(i-1, j)$ or the cell to the left $(i, j-1)$. The minimum sum to reach (i, j) is the value at (i, j) plus the smaller of the two incoming paths.

Analogy: Imagine you're walking from the top-left to bottom-right of a field where each cell has a toll cost. You can only move right or down. At each cell, to minimise the total toll paid so far, you'd choose the cheaper of the two roads that brought you there.

Step-by-step reasoning:

1. The starting cell $(0, 0)$ has path sum equal to its own value.
2. For the first row, each cell can only come from the left: $dp[0][j] = \text{grid}[0][j] + dp[0][j-1]$.
3. For the first column, each cell can only come from above: $dp[i][0] = \text{grid}[i][0] + dp[i-1][0]$.
4. For all other cells, choose the smaller incoming path: $dp[i][j] = \text{grid}[i][j] + \min(dp[i-1][j], dp[i][j-1])$.

Why it works: The path to each sub-cell (i, j) is independent of future choices (optimal substructure). The overlapping subproblems are the $dp[i-1][j]$ and $dp[i][j-1]$ values which are reused for multiple later cells.

3. When to Use It

- 2D grid with **right/down** movement constraints.
- Problems asking for **minimum or maximum sum path**.
- Problems where each cell has a **cost or value** and you need an optimal path.
- Any grid traversal where the decision at each cell is based on incoming neighbours.

Trigger phrases: "Minimum path sum", "maximum path sum", "grid traversal", "top-left to bottom-right", "move only right and down", "minimum cost to reach".

4. When Not to Use It

- When **diagonal movement** is allowed — still works with 3 incoming directions (top, left, top-left diagonal).
- When **all four directions** are allowed — becomes shortest path in a weighted graph (Dijkstra or BFS with 0-1 BFS if unweighted).
- When there are **negative cycles** or obstacles that make greedy choices invalid — may need Bellman-Ford or DP with obstacles.
- When the grid is **very large** ($10^5 \times 10^5$) — can't store the full DP matrix; may need different approach.
- When the path can **start and end at arbitrary cells** (not just corners) — different DP formulation.

5. Core Concepts

Concept	Explanation	Why It Matters
State	<code>dp[i][j] = min/max sum to reach (i, j)</code>	Foundation of the DP
Transition	<code>dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])</code>	Captures the movement constraint
Base row/column	First row and column have only one incoming direction	Must initialise separately
In-place vs separate DP	Modify grid in-place or use a separate dp array	In-place saves memory but mutates input
Direction of movement	Right and down only	This constraint makes it a simple DP

6. Step-by-Step Algorithm

Minimum Path Sum (grid of size $m \times n$):

1. If grid is empty, return 0.
2. Let `m = rows`, `n = cols`.
3. Create dp array of size $m \times n$ (or use grid in-place).
4. `dp[0][0] = grid[0][0]`.
5. Fill first row: `dp[0][j] = grid[0][j] + dp[0][j-1]` for `j = 1..n-1`.
6. Fill first column: `dp[i][0] = grid[i][0] + dp[i-1][0]` for `i = 1..m-1`.
7. For `i = 1..m-1`, `j = 1..n-1`:
 - `dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])`.
8. Return `dp[m-1][n-1]`.

7. Dry Run

Input:

```
grid = [
  [1, 3, 1],
  [1, 5, 1],
  [4, 2, 1]
]
```

DP Table Construction:

Step	(0,0)	(0,1)	(0,2)	(1,0)	(1,1)	(1,2)	(2,0)	(2,1)	(2,2)
Init	1	-	-	-	-	-	-	-	-
Row 0	1	1+3=4	4+1=5	-	-	-	-	-	-
Col 0	1	4	5	1+1=2	-	-	2+4=6	-	-
(1,1)	1	4	5	2	5+min(2,4)=7	-	6	-	-
(1,2)	1	4	5	2	7	1+min(7,5)=6	6	-	-
(2,1)	1	4	5	2	7	6	6	2+min(6,6)=8	-
(2,2)	1	4	5	2	7	6	6	8	1+min(8,6)=7

Answer: 7 (Path: 1 → 3 → 1 → 1 → 1, sum = 7)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Minimum Path Sum -----
// LeetCode 64
int minPathSum(vector<vector<int>>& grid) {
    if (grid.empty() || grid[0].empty()) return 0;

    int m = grid.size();
    int n = grid[0].size();

    // Use a separate DP table (or modify grid in-place)
    vector<vector<int>> dp(m, vector<int>(n));

    dp[0][0] = grid[0][0];

    // Fill first row
    for (int j = 1; j < n; ++j)
        dp[0][j] = grid[0][j] + dp[0][j - 1];

    // Fill first column
    for (int i = 1; i < m; ++i)
        dp[i][0] = grid[i][0] + dp[i - 1][0];

    // Fill rest
    for (int i = 1; i < m; ++i) {
```

```

        for (int j = 1; j < n; ++j) {
            dp[i][j] = grid[i][j] + min(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    return dp[m - 1][n - 1];
}

// ----- In-place version (saves memory) -----
int minPathSumInPlace(vector<vector<int>>& grid) {
    if (grid.empty() || grid[0].empty()) return 0;

    int m = grid.size(), n = grid[0].size();

    // First row
    for (int j = 1; j < n; ++j)
        grid[0][j] += grid[0][j - 1];

    // First column
    for (int i = 1; i < m; ++i)
        grid[i][0] += grid[i - 1][0];

    // Rest
    for (int i = 1; i < m; ++i)
        for (int j = 1; j < n; ++j)
            grid[i][j] += min(grid[i - 1][j], grid[i][j - 1]);

    return grid[m - 1][n - 1];
}

// ----- Maximum Path Sum (same idea, replace min with max) -----
int maxPathSum(vector<vector<int>>& grid) {
    if (grid.empty() || grid[0].empty()) return 0;

    int m = grid.size(), n = grid[0].size();
    vector<vector<int>> dp(m, vector<int>(n));

    dp[0][0] = grid[0][0];
    for (int j = 1; j < n; ++j)
        dp[0][j] = grid[0][j] + dp[0][j - 1];
    for (int i = 1; i < m; ++i)
        dp[i][0] = grid[i][0] + dp[i - 1][0];

    for (int i = 1; i < m; ++i)
        for (int j = 1; j < n; ++j)
            dp[i][j] = grid[i][j] + max(dp[i - 1][j], dp[i][j - 1]);

    return dp[m - 1][n - 1];
}

// ----- Example usage -----
int main() {
    vector<vector<int>> grid = {
        {1, 3, 1},
        {1, 5, 1},
        {4, 2, 1}
    };

    cout << "Min path sum: " << minPathSum(grid) << "\n"; // 7
}

```

```

    return 0;
}

```

9. Python Implementation

```

# ----- Minimum Path Sum -----
def min_path_sum(grid: list[list[int]]) -> int:
    if not grid or not grid[0]:
        return 0

    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]

    dp[0][0] = grid[0][0]

    # First row
    for j in range(1, n):
        dp[0][j] = grid[0][j] + dp[0][j - 1]

    # First column
    for i in range(1, m):
        dp[i][0] = grid[i][0] + dp[i - 1][0]

    # Rest
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i - 1][j], dp[i][j - 1])

    return dp[m - 1][n - 1]

# ----- In-place version -----
def min_path_sum_inplace(grid: list[list[int]]) -> int:
    if not grid or not grid[0]:
        return 0

    m, n = len(grid), len(grid[0])

    for j in range(1, n):
        grid[0][j] += grid[0][j - 1]
    for i in range(1, m):
        grid[i][0] += grid[i - 1][0]
    for i in range(1, m):
        for j in range(1, n):
            grid[i][j] += min(grid[i - 1][j], grid[i][j - 1])

    return grid[m - 1][n - 1]

# ----- Maximum Path Sum -----
def max_path_sum(grid: list[list[int]]) -> int:
    if not grid or not grid[0]:
        return 0

    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]

    dp[0][0] = grid[0][0]

```

```

    for j in range(1, n):
        dp[0][j] = grid[0][j] + dp[0][j - 1]
    for i in range(1, m):
        dp[i][0] = grid[i][0] + dp[i - 1][0]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + max(dp[i - 1][j], dp[i][j - 1])

    return dp[m - 1][n - 1]

# Example
grid = [
    [1, 3, 1],
    [1, 5, 1],
    [4, 2, 1]
]
print(min_path_sum(grid)) # 7

```

10. Code Explanation

- **Empty check:** `if (grid.empty() || grid[0].empty()) return 0;` handles empty input.
- **DP table initialisation:** A separate `dp` array avoids mutating input. In-place version modifies `grid` directly.
- **First row special case:** Since you can only come from the left, `dp[0][j] = grid[0][j] + dp[0][j-1]` is a simple prefix sum.
- **First column special case:** Since you can only come from above, `dp[i][0] = grid[i][0] + dp[i-1][0]`.
- **Main recurrence:** `dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])` — adds current cell's value to the cheaper incoming path.
- **Return value:** `dp[m-1][n-1]` holds the minimum sum to reach the bottom-right.
- **Max version:** Identical except `min` → `max`.

11. Complexity Analysis

Variant	Time	Space	Notes
Separate DP table	$O(m \times n)$	$O(m \times n)$	Easy to understand
In-place	$O(m \times n)$	$O(1)$ extra	Mutates input — may not be allowed
1D DP (space optimised)	$O(m \times n)$	$O(n)$	Uses single row — see variation
Max path sum	$O(m \times n)$	$O(m \times n)$	Same complexity

12. Common Patterns

Pattern	Identification	Approach	Example
Min path sum	"Minimum cost from top-left to bottom-right"	<code>dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])</code>	LeetCode 64
Max path sum	"Maximum sum path"	Same with <code>max()</code>	LeetCode (variation)
Triangle min path	"Min sum from top to bottom of triangle"	<code>dp[i][j] = grid[i][j] + min(dp[i-1][j-1], dp[i-1][j])</code>	LeetCode 120
With obstacles	"Some cells are blocked"	If blocked, <code>dp[i][j] = INF</code>	LeetCode 63 (Unique Paths II)

Pattern	Identification	Approach	Example
With diagonal moves	"Can also move diagonally"	$dp[i][j] = grid[i][j] + \min(3 \text{ directions})$	Variation

13. Common Mistakes

- **Off-by-one in indices:** Forgetting that `dp[0][0]` is the start, not `dp[1][1]`.
- **Not initialising first row/column separately:** If you use the general recurrence on row 0, `dp[i-1][j]` would access `dp[-1][j]`.
- **Using INT_MAX for min path: Need to initialise with large values but then handle base case correctly.**
- **Forgetting that grid values may be negative:** Min path with negatives still works with the same recurrence.
- **Overcomplicating when 1D space optimisation is needed:** Interviewers rarely ask for 1D DP optimisation for path sum, but it's possible.
- **Not checking empty grid:** Crashes on `grid[0].size()`.

14. Edge Cases

Case	Input	Expected	Reason
Empty grid	<code>[]</code>	0	Handle gracefully
Single cell	<code>[[5]]</code>	5	Start = end
Single row	<code>[[1, 2, 3]]</code>	6 (1+2+3)	Can only move right
Single column	<code>[[1], [2], [3]]</code>	6 (1+2+3)	Can only move down
All negatives	<code>[[-1, -2], [-3, -4]]</code>	-7	Min picks most negative
Large values	Up to 10^9	Sum may overflow int	Use long long

15. Variations

Variation	Description	Importance
Triangle (LeetCode 120)	Triangle grid, min top-to-bottom path	High
Dungeon Game (LeetCode 174)	Need minimum health to reach end with +ve/-ve cells	High — reverse DP
Cherry Pickup (LeetCode 741)	Two passes (forward + backward)	Hard
Minimum Falling Path Sum (LeetCode 931)	Can fall from any cell in top row to bottom	Medium
Path with obstacles	Some cells blocked = INF	Medium (Unique Paths II)

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Unique Paths	Same grid structure, counting instead of sum	Count vs min/max

Algorithm	Connection	How to Choose
Dijkstra	Both find min cost path	Dijkstra for 4-direction movement, DP for right/down
Bellman-Ford	Handles negative weights	Not needed for DAG-like grids (right/down = DAG)
A*	Heuristic-based pathfinding	Overkill for right/down grid

17. Practice Problems

Easy

1. **Minimum Path Sum** — LeetCode 64 — Direct problem — Medium (Easy as concept)
2. **Maximum Path Sum (GFG)** — Same with max — Easy

Medium

1. **Triangle** — LeetCode 120 — Triangle grid — Medium
2. **Minimum Falling Path Sum** — LeetCode 931 — Vertical movement — Medium
3. **Unique Paths II** — LeetCode 63 — With obstacles — Medium

Hard

1. **Dungeon Game** — LeetCode 174 — Reverse DP, health calculation — Hard
2. **Cherry Pickup** — LeetCode 741 — Two-pass DP — Hard

18. Interview Explanation

"Minimum Path Sum is a classic grid DP problem. Since you can only move right or down, the grid is a DAG — there are no cycles. This means the path to any cell only depends on the paths to the cell above and the cell to the left. I build a DP table where $dp[i][j]$ = minimum sum to reach (i, j) . The recurrence is $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$. I initialise the first row as a prefix sum and the first column similarly, since they only have one incoming direction. Time and space are $O(m \times n)$. I can optimise space to $O(n)$ by keeping only the current and previous rows, or modify the grid in-place if allowed."

19. Revision Notes

- **Key idea:** $dp[i][j] = \text{value} + \min(\text{top}, \text{left})$.
- **Formula:** $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$.
- **Initialisation:** First row = prefix sum, first column = prefix sum.
- **Template:** Fill row 0, col 0, then double loop with min/max.
- **Complexity:** $O(m \times n)$ time, $O(m \times n)$ or $O(n)$ space.
- **Traps:** Empty grid, negative values, off-by-one in indices.

20. Final Cheat Sheet

MIN/MAX PATH SUM

When to use: 2D grid, right/down moves, min or max sum
 Recurrence: $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$
 Initialisation: $dp[0][0] = grid[0][0]$; first row/col = prefix sums
 Time: $O(m \times n)$

Space: $O(m \times n)$ or $O(n)$ with 1D DP

Key code:

```
dp[0][0] = grid[0][0];
for (j = 1..n-1) dp[0][j] = grid[0][j] + dp[0][j-1];
for (i = 1..m-1) dp[i][0] = grid[i][0] + dp[i-1][0];
for (i = 1..m-1)
    for (j = 1..n-1)
        dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]);
return dp[m-1][n-1];
```

Edge cases: Empty grid, single row/col, negative values

Variations: Triangle, max path, obstacles, dungeon game

5. GRID DP

1. Overview

Grid DP is the general framework for solving problems on a 2D grid using dynamic programming. It covers counting paths (Unique Paths), minimising/maximising sums (Path Sum), and problems with obstacles, varying movement rules, or multiple passes. The core idea is that the state is defined by grid coordinates (i, j) , and transitions come from neighbours you can move from.

2. Intuition

Simple explanation: Any problem on a grid where you move step-by-step from one cell to an adjacent cell, and the decision at each cell depends on previous cells, can be solved with grid DP. You build a table where each cell (i, j) stores the answer for reaching that cell.

Analogy: Think of filling a spreadsheet where each cell's formula references the cell above and the cell to the left. Once all cells are filled, the answer is in the bottom-right cell.

Step-by-step reasoning:

1. Define what $dp[i][j]$ represents (ways, min cost, max sum, boolean, etc.).
2. Define the movement rule (right/down, all 4 directions, etc.).
3. Initialise base cases (start cell, first row, first column).
4. Fill the grid in an order that respects dependencies (top-left to bottom-right for right/down moves).
5. Return the answer from the target cell.

Why it works: The grid traversal (right/down only) naturally forms a DAG. When traversal is top-left to bottom-right, every cell's dependencies (above, left) are already computed. This guarantees correct bottom-up DP.

3. When to Use It

- Any problem on a 2D grid with **directed acyclic movement**.
- Problems involving **paths, sums, or decisions** at each cell.
- Problems where each cell's result depends on **neighbouring cells**.
- Counting, minimising, maximising, or boolean feasibility on a grid.

Trigger phrases: "Grid", "matrix", "2D array", "path in grid", "move right/down", "reach bottom-right", "number of ways to reach", "minimum cost to travel".

4. When Not to Use It

- When the grid is **extremely large** ($10^5 \times 10^5$) — DP table won't fit in memory. Look for combinatorial formulas.
- When movement is **any of 4 directions with cycles** — use Dijkstra or BFS instead.
- When obstacles change dynamically — DP requires static grid.
- When you need to **reconstruct all paths** (not just count) — backtracking with DP works but is expensive.
- When the problem is better solved with **graph algorithms** (e.g., shortest path in weighted grid with 4-direction movement).

5. Core Concepts

Concept	Explanation	Why It Matters
State (i, j)	Current position in the grid	Defines the DP table dimensions
Movement rule	Which neighbours can be reached	Determines transition direction
Base cells	Start cell, edges of the grid	Foundation of the recurrence
Filling order	Top-to-bottom, left-to-right	Ensures dependencies are computed first
Space optimisation	1D array for row-by-row fill	Reduces memory from $O(mn)$ to $O(n)$

6. Step-by-Step Algorithm

General Grid DP (right/down moves):

1. Get `m = rows, n = cols`.
2. Create `dp` table of size `m × n` (or 1D of size `n` for space optimisation).
3. Initialise `dp[0][0]` based on the problem (count = 1, cost = `grid[0][0]`, etc.).
4. Fill base row and column.
5. For `i = 1..m-1`, for `j = 1..n-1`:
 - `dp[i][j] = f(dp[i-1][j], dp[i][j-1])` where `f` is problem-specific.
6. Return `dp[m-1][n-1]`.

7. Dry Run

Same as Min Path Sum (Section 4) or Unique Paths (Section 6) — Grid DP is the umbrella topic; specific dry runs are under each sub-problem.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Generic Grid DP Framework -----
// This is a template. Replace `combine` and `identity` as needed.

template<typename T>
T gridDP(vector<vector<T>>& grid,
         T (*combine)(T, T),
         T identity) {
    if (grid.empty() || grid[0].empty()) return identity;
```

```

int m = grid.size(), n = grid[0].size();
vector<vector<T>> dp(m, vector<T>(n));

dp[0][0] = grid[0][0]; // or 1 for counting

// First row
for (int j = 1; j < n; ++j)
    dp[0][j] = combine(dp[0][j - 1], identity) + grid[0][j]; // simplified; adjust per
problem

// First column
for (int i = 1; i < m; ++i)
    dp[i][0] = combine(dp[i - 1][0], identity) + grid[i][0];

// Rest
for (int i = 1; i < m; ++i)
    for (int j = 1; j < n; ++j)
        dp[i][j] = grid[i][j] + combine(dp[i - 1][j], dp[i][j - 1]);

return dp[m - 1][n - 1];
}

// ----- Space Optimised Grid DP (1D array) -----
// For problems where dp[i][j] depends only on dp[i-1][j] and dp[i][j-1]
int gridDPSpaceOptimised(vector<vector<int>>& grid, bool useMin = true) {
    if (grid.empty() || grid[0].empty()) return 0;

    int m = grid.size(), n = grid[0].size();
    vector<int> dp(n);

    // First row
    dp[0] = grid[0][0];
    for (int j = 1; j < n; ++j)
        dp[j] = grid[0][j] + dp[j - 1];

    // Remaining rows
    for (int i = 1; i < m; ++i) {
        dp[0] += grid[i][0]; // first column: only from above
        for (int j = 1; j < n; ++j) {
            if (useMin)
                dp[j] = grid[i][j] + min(dp[j], dp[j - 1]);
            else
                dp[j] = grid[i][j] + max(dp[j], dp[j - 1]);
        }
    }

    return dp[n - 1];
}

// ----- Example: min path sum using 1D DP -----
int minPathSum1D(vector<vector<int>>& grid) {
    return gridDPSpaceOptimised(grid, true);
}

int main() {
    vector<vector<int>> grid = {
        {1, 3, 1},
        {1, 5, 1},
    }
}

```

```

        {4, 2, 1}
    };
    cout << "Min path sum (1D DP): " << minPathSum1D(grid) << "\n"; // 7
    return 0;
}

```

9. Python Implementation

```

# ----- Space Optimised Grid DP (1D array) -----
def grid_dp_1d(grid: list[list[int]], use_min: bool = True) -> int:
    if not grid or not grid[0]:
        return 0

    m, n = len(grid), len(grid[0])
    dp = [0] * n

    # First row
    dp[0] = grid[0][0]
    for j in range(1, n):
        dp[j] = grid[0][j] + dp[j - 1]

    # Remaining rows
    for i in range(1, m):
        dp[0] += grid[i][0] # first column: only from above
        for j in range(1, n):
            if use_min:
                dp[j] = grid[i][j] + min(dp[j], dp[j - 1])
            else:
                dp[j] = grid[i][j] + max(dp[j], dp[j - 1])

    return dp[n - 1]

# ----- Generic Grid DP -----
def grid_dp(grid: list[list[int]], combine_func, identity: int) -> int:
    if not grid or not grid[0]:
        return identity

    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]

    dp[0][0] = grid[0][0]
    for j in range(1, n):
        dp[0][j] = combine_func(dp[0][j - 1], identity) + grid[0][j]
    for i in range(1, m):
        dp[i][0] = combine_func(dp[i - 1][0], identity) + grid[i][0]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + combine_func(dp[i - 1][j], dp[i][j - 1])

    return dp[m - 1][n - 1]

# Example
grid = [
    [1, 3, 1],
    [1, 5, 1],
    [4, 2, 1]
]

```

```
]
print(grid_dp_1d(grid, use_min=True)) # 7
```

10. Code Explanation

- **1D DP (space optimisation):** Instead of a 2D table, we use a 1D array of size n . At row i , $dp[j]$ represents the answer for (i, j) . When we process row i , $dp[j]$ still holds the value for $(i-1, j)$ (from the previous row), and $dp[j-1]$ now holds the value for $(i, j-1)$ (already updated in this row). This works because the recurrence only needs **top** (previous row, same column) and **left** (current row, previous column).
- **First row handling:** $dp[j] = grid[0][j] + dp[j-1]$ — only left neighbour exists.
- **First column handling:** $dp[0] += grid[i][0]$ — only top neighbour exists.
- **Combine function:** **min** or **max** depending on problem (min path sum, max path sum).
- **Generic template:** The `gridDP` function accepts a combine function pointer, making it reusable across problems.

11. Complexity Analysis

Variant	Time	Space	Notes
2D DP table	$O(m \times n)$	$O(m \times n)$	Simple, easy to debug
1D DP (row optimisation)	$O(m \times n)$	$O(n)$	Best for most cases
1D DP (min/max only)	$O(m \times n)$	$O(1)$ in-place	Mutates input
With path reconstruction	$O(m \times n)$	$O(m \times n)$	Need parent pointers

12. Common Patterns

Pattern	Identification	Approach
Counting paths	"Number of ways to reach bottom-right"	$dp[i][j] = dp[i-1][j] + dp[i][j-1]$
Min path sum	"Minimum cost to reach bottom-right"	$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$
Max path sum	"Maximum sum path"	$dp[i][j] = grid[i][j] + \max(dp[i-1][j], dp[i][j-1])$
With obstacles	"Some cells are blocked"	If blocked: $dp[i][j] = 0$ or INF
With variable values	"Each cell has a profit/cost"	Combine according to problem
Feasibility (bool)	"Is there a path satisfying condition?"	$dp[i][j] = dp[i-1][j]$ OR $dp[i][j-1]$

13. Common Mistakes

- **Wrong filling order:** When moves are right/down, fill top-to-bottom, left-to-right. For other moves, adjust.
- **Not handling obstacles:** For blocked cells, skip the transition ($dp[i][j] = 0$ or INF).
- **Overflow in counting problems:** Number of paths grows exponentially. Use **long long** or mod.
- **Confusing row and column indices:** $grid[i][j]$ where i = row, j = column.
- **1D DP update order:** Must update $dp[j]$ left to right so $dp[j-1]$ is the current row's left value.
- **Empty grid not checked.**

14. Edge Cases

Case	Input	Expected	Reason
Empty grid	<code>[]</code>	0 or identity	Guard

Case	Input	Expected	Reason
1×1 grid	[[5]]	5	Start = end
1×n grid	[[1,2,3]]	Prefix sum path	Only right
m×1 grid	[[1],[2],[3]]	Prefix sum path	Only down
All blocked	Obstacle at start	0 (no ways)	Unreachable
Large grid	1000×1000	Various	Need 1D DP or mod

15. Variations

Variation	What Changes	Importance
Unique Paths	Count paths, no weights	High — LeetCode 62
Unique Paths II	With obstacles	High — LeetCode 63
Min Path Sum	Weighted cells, min sum	High — LeetCode 64
Triangle	Triangular grid	High — LeetCode 120
Cherry Pickup	Two passes (round trip)	Hard — LeetCode 741
Dungeon Game	Reverse DP with health	Hard — LeetCode 174
Constrained Subsequence	Not actually grid but 2D DP	Medium

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Unique Paths	Special case of grid DP (counting)	Use combinatorial formula for small constraints
Shortest Path in DAG	Grid with right/down = DAG	Same DP concept
Floyd-Warshall	All-pairs shortest on grid	Overkill for grid DP
Dijkstra	4-direction movement	Use when moves are all 4 directions

17. Practice Problems

Easy

1. **Unique Paths** — LeetCode 62 — Counting paths — Medium (Easy conceptually)
2. **Minimum Path Sum** — LeetCode 64 — Min weighted path — Medium

Medium

1. **Unique Paths II** — LeetCode 63 — With obstacles — Medium
2. **Triangle** — LeetCode 120 — Triangle grid path — Medium
3. **Minimum Falling Path Sum** — LeetCode 931 — Fall from any top cell — Medium

Hard

1. **Dungeon Game** — LeetCode 174 — Reverse DP — Hard
2. **Cherry Pickup** — LeetCode 741 — Two-pass DP — Hard

18. Interview Explanation

"Grid DP is a versatile framework for problems on a 2D grid with directed movement. The core idea is that each cell's answer depends on answers from neighbouring cells (above, left, etc.). Since movement is typically restricted to right and down, the grid forms a DAG and we fill it top-left to bottom-right. The state is simply the coordinates (i, j) . Transitions are problem-specific: addition for counting, min/max for optimisation, or boolean operations for feasibility. I usually implement it with a 1D array for space optimisation — $O(n)$ instead of $O(m \times n)$ — by updating the array left to right so $dp[j]$ represents the current row and $dp[j-1]$ the left neighbour. The key steps are: initialise the top row and left column, then iterate row by row applying the problem's recurrence."

19. Revision Notes

- **Key idea:** $dp[i][j]$ depends on neighbour cells (top, left); fill top-left to bottom-right.
- **General recurrence:** $dp[i][j] = f(dp[i-1][j], dp[i][j-1])$.
- **Space optimisation:** 1D array of size n , update left to right.
- **Common problems:** Counting paths (add), min/max path sum (min/max), obstacles (skip).
- **Complexity:** $O(m \times n)$ time, $O(n)$ space (1D) or $O(1)$ in-place.
- **Traps:** Wrong fill order, not handling obstacles, overflow in counting, row vs column indices.

20. Final Cheat Sheet

GRID DP

When to use: 2D grid with directed movement (right/down)

State: $dp[i][j]$ = answer for cell (i, j)

Recurrence: $dp[i][j] = f(dp[i-1][j], dp[i][j-1])$

Fill order: Top to bottom, left to right

Time: $O(m \times n)$

Space: $O(n)$ with 1D array, $O(m \times n)$ full table

Key code (1D):

```
dp[0] = start_value;
for j = 1..n-1: dp[j] = grid[0][j] + dp[j-1];
for i = 1..m-1:
    dp[0] += grid[i][0];
    for j = 1..n-1:
        dp[j] = grid[i][j] + combine(dp[j], dp[j-1]);
return dp[n-1];
```

Edge cases: Empty grid, 1×1 , $1 \times n$, $m \times 1$, obstacles

Variations: Counting, min/max, obstacles, triangle, dungeon

6. UNIQUE PATHS

1. Overview

A robot is located at the top-left corner of an $m \times n$ grid. It can only move **right** or **down**. How many **unique paths** are there to reach the bottom-right corner? This is the classic combinatorics-cum-DP problem. The answer is $C(m+n-2, m-1)$ (or $C(m+n-2, n-1)$) but DP provides a simple recurrence: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ with $dp[0][j] = 1$ and $dp[i][0] = 1$.

2. Intuition

Simple explanation: To reach cell (i, j) , you must come from $(i-1, j)$ (down move) or $(i, j-1)$ (right move). The number of unique paths to (i, j) is the sum of the number of unique paths to these two predecessor cells.

Analogy: Think of Pascal's triangle laid flat. The number at each position is the sum of the number above and to the left. Unique Paths is literally Pascal's triangle embedded in a grid.

Step-by-step reasoning:

1. There is exactly 1 way to reach any cell in the first row (always move right).
2. There is exactly 1 way to reach any cell in the first column (always move down).
3. For any other cell (i, j) , the number of ways = ways to reach above cell + ways to reach left cell.
4. By the time you reach bottom-right, you have the total number of unique paths.

Why it works: The path choices (right or down) are independent and cumulative. Each path is a sequence of $(m-1)$ down moves and $(n-1)$ right moves in some order. The number of distinct sequences = number of ways to choose positions of down moves among $(m+n-2)$ total moves = $C(m+n-2, m-1)$.

3. When to Use It

- Counting the number of paths from top-left to bottom-right with right/down moves.
- Any problem where the answer is the binomial coefficient $C(m+n-2, m-1)$.
- Subproblems of larger grid DP problems (e.g., with obstacles first solve base unique paths).
- As a building block for more complex grid counting.

Trigger phrases: "Robot in a grid", "unique paths", "number of ways to reach bottom-right", "top-left to bottom-right", "move only right and down".

4. When Not to Use It

- When moves include **up or left** (cycles) — not a DAG; needs different DP or BFS.
- When the grid has **weights or costs** — becomes min/max path sum, not counting.
- When there are **obstacles** — use Unique Paths II (same recurrence but skip blocked cells).
- When m and n are very large ($m, n > 1000$) and modulo is not asked — the number of paths grows astronomically (overflows 64-bit at around $m=n=20$). Use `long long` or modulo.
- When the robot can move in **more than 2 directions** (e.g., also diagonally) — different recurrence.

5. Core Concepts

Concept	Explanation	Why It Matters
State	$dp[i][j]$ = unique paths to (i, j)	Core DP value
Transition	$dp[i][j] = dp[i-1][j] + dp[i][j-1]$	Direct additive recurrence
Base values	First row = 1, first column = 1	Only one way to reach border cells
Combinatorial formula	$C(m+n-2, m-1)$	$O(\min(m,n))$ time, $O(1)$ space; useful for pure counting
Space optimisation	$dp[j] += dp[j-1]$ (1D array)	$O(n)$ space instead of $O(mn)$

6. Step-by-Step Algorithm

DP Approach:

1. If $m == 0$ or $n == 0$, return 0.
2. Create `dp` array of size n (1D optimised).
3. Fill `dp` with 1 (first row: all 1 way).
4. For $i = 1$ to $m-1$:
 - For $j = 1$ to $n-1$:
 - $dp[j] = dp[j] + dp[j-1]$ ($dp[j]$ is the value above, $dp[j-1]$ is the value to the left).
5. Return $dp[n-1]$.

Combinatorial Approach:

1. Total moves = $m + n - 2$.
2. Choose positions for down moves: $C(\text{total}, m - 1)$.
3. Compute efficiently using a loop (multiplication and division).

7. Dry Run

Input: $m = 3, n = 4$

DP Table (full 2D):

(i\j)	0	1	2	3
0	1	1	1	1
1	1	2	3	4
2	1	3	6	10

1D DP progression:

- Initial `dp` = [1, 1, 1, 1] (row 0)
- After row 1: `dp` = [1, 2, 3, 4]
- After row 2: `dp` = [1, 3, 6, 10]

Answer: 10

Verification using combinatorics: $C(3+4-2, 3-1) = C(5, 2) = 10$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Unique Paths: 1D DP -----
// LeetCode 62
int uniquePaths(int m, int n) {
    // 1D DP array
    vector<long long> dp(n, 1);

    for (int i = 1; i < m; ++i) {
        for (int j = 1; j < n; ++j) {
            // dp[j] (above) + dp[j-1] (left)
            dp[j] = dp[j] + dp[j - 1];
        }
    }
}
```

```

    }
    return dp[n - 1];
}

// ----- Unique Paths: 2D DP (clearer) -----
int uniquePaths2D(int m, int n) {
    vector<vector<int>> dp(m, vector<int>(n, 1));

    for (int i = 1; i < m; ++i) {
        for (int j = 1; j < n; ++j) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
    return dp[m - 1][n - 1];
}

// ----- Unique Paths: Combinatorial (O(min(m,n))) -----
long long nCr(int n, int r) {
    if (r > n - r) r = n - r; // Use smaller r
    long long res = 1;
    for (int i = 0; i < r; ++i) {
        res = res * (n - i) / (i + 1);
    }
    return res;
}

int uniquePathsCombinatorial(int m, int n) {
    // C(m+n-2, m-1) or C(m+n-2, n-1)
    return nCr(m + n - 2, m - 1);
}

// ----- Unique Paths II: With obstacles -----
// LeetCode 63
int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
    if (obstacleGrid.empty() || obstacleGrid[0].empty()) return 0;

    int m = obstacleGrid.size(), n = obstacleGrid[0].size();
    vector<long long> dp(n, 0);

    // Initialise first cell
    dp[0] = (obstacleGrid[0][0] == 0) ? 1 : 0;

    // First row
    for (int j = 1; j < n; ++j) {
        if (obstacleGrid[0][j] == 0)
            dp[j] = dp[j - 1]; // only from left
        else
            dp[j] = 0;
    }

    // Rest
    for (int i = 1; i < m; ++i) {
        // First column
        if (obstacleGrid[i][0] == 1)
            dp[0] = 0;
        // (else dp[0] stays as is from row above)

        for (int j = 1; j < n; ++j) {

```

```

        if (obstacleGrid[i][j] == 1)
            dp[j] = 0;
        else
            dp[j] = dp[j] + dp[j - 1];
    }
}

return dp[n - 1];
}

// ----- Example usage -----
int main() {
    cout << "3x4 grid: " << uniquePaths(3, 4) << "\n";           // 10
    cout << "3x4 grid (combinatorial): " << uniquePathsCombinatorial(3, 4) << "\n"; // 10

    vector<vector<int>> obstacles = {
        {0, 0, 0},
        {0, 1, 0},
        {0, 0, 0}
    };
    cout << "With obstacles: " << uniquePathsWithObstacles(obstacles) << "\n"; // 2
    return 0;
}

```

9. Python Implementation

```

# ----- Unique Paths: 1D DP -----
def unique_paths(m: int, n: int) -> int:
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]
    return dp[n - 1]

# ----- Unique Paths: 2D DP -----
def unique_paths_2d(m: int, n: int) -> int:
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]

# ----- Unique Paths: Combinatorial -----
import math
def unique_paths_combinatorial(m: int, n: int) -> int:
    return math.comb(m + n - 2, m - 1)

# ----- Unique Paths II: With obstacles -----
def unique_paths_with_obstacles(obstacleGrid: list[list[int]]) -> int:
    if not obstacleGrid or not obstacleGrid[0]:
        return 0

    m, n = len(obstacleGrid), len(obstacleGrid[0])
    dp = [0] * n

    dp[0] = 1 if obstacleGrid[0][0] == 0 else 0

```

```

    for j in range(1, n):
        dp[j] = dp[j - 1] if obstacleGrid[0][j] == 0 else 0

    for i in range(1, m):
        if obstacleGrid[i][0] == 1:
            dp[0] = 0
        for j in range(1, n):
            if obstacleGrid[i][j] == 1:
                dp[j] = 0
            else:
                dp[j] += dp[j - 1]

    return dp[n - 1]

# Example
print(unique_paths(3, 4))  # 10

```

10. Code Explanation

- **1D DP approach:** $dp[j]$ initially represents row 0 (all 1s). For each subsequent row, $dp[j] = dp[j] + dp[j-1]$. Here, $dp[j]$ (before update) is the value from the cell above, and $dp[j-1]$ (already updated) is the value from the cell to the left. This elegantly computes the recurrence with $O(n)$ space.
- **2D DP approach:** Uses full $m \times n$ table. $dp[0][j] = 1$ and $dp[i][0] = 1$ initialise borders. Then $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ fills the rest.
- **Combinatorial approach:** $C(m+n-2, m-1)$ counts ways to arrange the down moves. The `nCr` function uses multiplication and division in one loop to avoid overflow. Use `long long` for intermediate values.
- **Unique Paths II (obstacles):** If a cell is an obstacle, $dp[j] = 0$. The first row and column handle obstacles individually. The recurrence becomes $dp[j] = dp[j] + dp[j-1]$ only when the cell is free.

11. Complexity Analysis

Variant	Time	Space	Notes
2D DP	$O(m \times n)$	$O(m \times n)$	Simple, good for interviews
1D DP	$O(m \times n)$	$O(n)$	Space-optimised best choice
Combinatorial	$O(\min(m, n))$	$O(1)$	No overflow protection needed; best for CP
With obstacles	$O(m \times n)$	$O(n)$ or $O(m)$	Same as 1D DP

12. Common Patterns

Pattern	Identification	Approach	Example
Standard unique paths	$m \times n$ grid, right/down only	$dp[j] += dp[j-1]$	LeetCode 62
With obstacles	Grid has 1s as blocked cells	Set $dp = 0$ for blocked cells	LeetCode 63
With variable start/end	Not top-left or bottom-right	Set $dp[start] = 1$, return $dp[end]$	Variation

Pattern	Identification	Approach	Example
Min/max with obstacles	Combination of obstacles + weights	Combine min/max with obstacle check	Variation
Paths with diagonals	Also diagonal moves	$dp[i][j] = dp[i-1][j] + dp[i][j-1] + dp[i-1][j-1]$	Variation

13. Common Mistakes

- **Forgetting overflow:** Count grows as binomial coefficients. `uniquePaths(20, 20) = 35345263800` — exceeds 32-bit `int`. Use `long long`.
- **Off-by-one with m and n:** A 1×1 grid has 1 path. A 1×5 grid has 1 path (all right moves). The code handles this correctly with `dp[0]` initialised to 1.
- **Blocked start or end in obstacle version:** If the start or end cell is an obstacle, return 0 immediately.
- **Combinatorial overflow in intermediate multiplication:** In `nCr`, multiply then divide to keep numbers manageable. `res = res * (n - i) / (i + 1)`.
- **Not handling empty grid:** Return 0 if `m == 0` or `n == 0`.

14. Edge Cases

Input (m, n)	Expected	Reason
(1, 1)	1	Start = end
(1, 5)	1	Only right moves
(5, 1)	1	Only down moves
(2, 2)	2	Right→Down, Down→Right
(3, 3)	6	$C(4, 2) = 6$
(10, 10)	48620	Large number
(19, 19)	35345263800	$> 2^{31}$ — need 64-bit

15. Variations

Variation	Description	Importance
Unique Paths II (obstacles)	Some cells blocked	High — LeetCode 63
Unique Paths III (exact cells)	Must visit all non-obstacle cells exactly once	Hard — LeetCode 980 (backtracking + DP)
Paths with diagonals	Also allow diagonal moves	Medium
Paths with maximum sum	Combine count with max sum	Medium
Paths with forbidden turns	Cannot turn twice consecutively	Hard

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Grid DP	Generalisation of unique paths	Use unique paths as building block

Algorithm	Connection	How to Choose
Pascal's Triangle	Same recurrence	Direct combinatorial formula
Combinatorics nCr	Closed-form solution	When m, n small and no obstacles
Min Path Sum	Weighted version of same grid	Add weights, replace sum with min/max

17. Practice Problems

Easy

1. **Unique Paths** — LeetCode 62 — Standard problem — Medium (Easy as DP)
2. **Unique Paths II** — LeetCode 63 — With obstacles — Medium

Medium

1. **Minimum Path Sum** — LeetCode 64 — Weighted version — Medium
2. **Unique Paths III** — LeetCode 980 — Exact visitation — Hard

Hard

1. **Cherry Pickup** — LeetCode 741 — Two-pass grid DP — Hard
2. **Dungeon Game** — LeetCode 174 — Reverse DP, health — Hard

18. Interview Explanation

"Unique Paths is a combinatorial counting problem, solvable both with DP and directly with combinatorics. The DP recurrence is simple: the number of ways to reach cell (i, j) equals the ways from above plus the ways from the left, since you can only move right or down. Both the first row and first column have exactly 1 way. I'd implement this with a 1D array of size n for $O(n)$ space, updating it left to right: $dp[j] += dp[j-1]$. If asked, I'd also mention the combinatorial solution: $C(m+n-2, m-1)$, which is $O(\min(m,n))$ time and $O(1)$ space, useful when m and n are small and there are no obstacles. For the follow-up with obstacles, I set $dp[j] = 0$ for blocked cells and avoid adding from blocked neighbours."

19. Revision Notes

- **Key idea:** Paths to (i,j) = paths from above + paths from left.
- **Formula (DP):** $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, base: all 1s in row 0 and col 0.
- **Formula (combinatorial):** $C(m+n-2, m-1)$.
- **Template (1D DP):** $dp = [1]*n$; for i in $1..m-1$: for j in $1..n-1$: $dp[j] += dp[j-1]$;
- **Complexity:** $O(mn)$ time, $O(n)$ space.
- **Traps:** Overflow (use long long), obstacles, empty grid, $m=1$ or $n=1$.

20. Final Cheat Sheet

UNIQUE PATHS

When to use: Count paths on grid with right/down moves only

Recurrence: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Base cases: First row = 1, first column = 1

Combinatorial: $C(m+n-2, m-1)$

Time: $O(m \times n)$ DP, $O(\min(m,n))$ combinatorial

Space: $O(n)$ 1D DP, $O(1)$ combinatorial

Key code (1D DP):

```
dp = vector<long long>(n, 1);
for (i = 1..m-1)
    for (j = 1..n-1)
        dp[j] += dp[j-1];
return dp[n-1];
```

Edge cases: $m=1$ or $n=1 \rightarrow 1$; obstacles $\rightarrow dp[j] = 0$ if blocked

Variations: Obstacles (II), must visit all (III), weights

7. KNAPSACK 0/1

1. Overview

The 0/1 Knapsack problem: Given n items, each with a **weight** $w[i]$ and a **value** $v[i]$, and a knapsack with capacity W , find the maximum value that can be carried. Each item can be taken **at most once** (hence "0/1"). This is the quintessential "decision DP" problem and the foundation for many resource-allocation DP problems.

2. Intuition

Simple explanation: For each item, you have two choices: take it or leave it. If you take it, you add its value and reduce the remaining capacity by its weight. If you leave it, the capacity and value stay unchanged. The optimal decision depends on which choice leads to higher total value.

Analogy: Imagine you're packing a suitcase for a trip with a weight limit. Each item has a weight and a usefulness score. For each item, you decide: "Is this item useful enough to take, given that it takes up space I might need for other items?" The DP explores all combinations efficiently.

Step-by-step reasoning:

1. Sort items (not necessary but helps intuition).
2. Consider items one by one.
3. For each item and each possible capacity, decide:
 - **Skip:** the best value with this capacity stays the same as without this item.
 - **Take:** add this item's value to the best value with capacity reduced by its weight.
4. Take the maximum of the two options.

Why it works: The optimal solution for capacity c using the first i items can be built from the optimal solution for smaller capacities using the first $i-1$ items (optimal substructure). Since each item is considered once, and all capacity values are explored, the DP explores all 2^n subsets implicitly without enumerating them.

3. When to Use It

- Selecting items with a **capacity/weight constraint** to maximise value.
- Problems where each item can be taken **at most once**.
- Resource allocation with limited budget/capacity.
- Problems that ask for "max value with given capacity".
- Subset sum problems (where values equal weights, or special case of 0/1 knapsack).

Trigger phrases: "Knapsack", "capacity", "weight", "maximum value", "0/1", "each item at most once", "choose items to maximize", "budget constraint".

4. When Not to Use It

- When items can be taken **multiple times** (unbounded knapsack) — use a different recurrence.
- When items are **fractionally divisible** (fractional knapsack) — greedy works (sort by value/weight ratio).
- When the capacity or number of items is **extremely large** ($W > 10^6$ or $n > 10^4$ with large W) — need meet-in-the-middle or branch-and-bound.
- When weights are **not integer** — DP on integers doesn't work directly; use meet-in-the-middle or greedy.
- When the goal is **minimum weight to achieve a certain value** — can invert the DP (DP on value instead of capacity).

5. Core Concepts

Concept	Explanation	Why It Matters
State	$dp[i][c]$ = max value using first i items with capacity c	2D DP table
Transition	$dp[i][c] = \max(dp[i-1][c], dp[i-1][c - w[i]] + v[i])$	Skip vs take
Capacity dimension	Iterate over all possible capacities from 1 to W	Ensures all subproblems are solved
Space optimisation	Use 1D array, iterate capacity descending	Critical for CP — $O(W)$ space
DP on value	Alternative: $dp[v]$ = min weight for value v	Useful when W is large but values are small

6. Step-by-Step Algorithm

Standard 0/1 Knapsack (2D DP):

1. Let $n = \text{items.size}()$, $W = \text{capacity}$.
2. Create dp of size $(n+1) \times (W+1)$, initialised to 0.
3. For $i = 1$ to n :
 - For $c = 1$ to W :
 - $\text{skip} = dp[i-1][c]$.
 - $\text{take} = 0$.
 - If $w[i-1] \leq c$: $\text{take} = dp[i-1][c - w[i-1]] + v[i-1]$.
 - $dp[i][c] = \max(\text{skip}, \text{take})$.
4. Return $dp[n][W]$.

Space Optimised (1D DP):

1. Create dp of size $W+1$, initialised to 0.
2. For $i = 0$ to $n-1$:
 - For $c = W$ down to $w[i]$:
 - $dp[c] = \max(dp[c], dp[c - w[i]] + v[i])$.
3. Return $dp[W]$.

Why descending order: In the 1D version, $dp[c - w[i]]$ must be the value **before** processing item i (i.e., $dp[i-1][c - w[i]]$). If we iterate capacity ascending, $dp[c - w[i]]$ would have already been updated for item i , allowing the item to be used multiple times (which is unbounded knapsack). Descending order prevents this.

7. Dry Run

Input:

- $w = [2, 3, 4, 5], v = [3, 4, 5, 6], W = 8$

2D DP Table:

i (item)	cap 0	cap 1	cap 2	cap 3	cap 4	cap 5	cap 6	cap 7	cap 8
0 (none)	0	0	0	0	0	0	0	0	0
1 (w=2,v=3)	0	0	3	3	3	3	3	3	3
2 (w=3,v=4)	0	0	3	4	4	7	7	7	7
3 (w=4,v=5)	0	0	3	4	5	7	8	9	9
4 (w=5,v=6)	0	0	3	4	5	7	8	9	10

Answer: 10 (items 2 and 4: weight 3+5=8, value 4+6=10)

1D DP Progression (descending capacity):

- Start: $dp = [0, 0, 0, 0, 0, 0, 0, 0, 0]$
- Item 1 (w=2, v=3): $dp = [0, 0, 3, 3, 3, 3, 3, 3, 3]$
- Item 2 (w=3, v=4): $dp = [0, 0, 3, 4, 4, 7, 7, 7, 7]$
- Item 3 (w=4, v=5): $dp = [0, 0, 3, 4, 5, 7, 8, 9, 9]$
- Item 4 (w=5, v=6): $dp = [0, 0, 3, 4, 5, 7, 8, 9, 10]$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- 0/1 Knapsack: 1D Space Optimised -----
int knapsack01(int W, vector<int>& weights, vector<int>& values) {
    int n = weights.size();
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        // Descending order to prevent reuse of same item
        for (int c = W; c >= weights[i]; --c) {
            dp[c] = max(dp[c], dp[c - weights[i]] + values[i]);
        }
    }
    return dp[W];
}

// ----- 0/1 Knapsack: 2D DP (for understanding) -----
int knapsack01_2D(int W, vector<int>& weights, vector<int>& values) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; ++i) {
        int w = weights[i - 1];
        int v = values[i - 1];
        for (int c = 1; c <= W; ++c) {
```

```

        int skip = dp[i - 1][c];
        int take = (w <= c) ? dp[i - 1][c - w] + v : 0;
        dp[i][c] = max(skip, take);
    }
}
return dp[n][W];
}

// ----- 0/1 Knapsack: Reconstruct selected items -----
vector<int> knapsackReconstruct(int W, vector<int>& weights,
                               vector<int>& values) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; ++i)
        for (int c = 1; c <= W; ++c) {
            int skip = dp[i - 1][c];
            int take = (weights[i - 1] <= c)
                ? dp[i - 1][c - weights[i - 1]] + values[i - 1]
                : 0;
            dp[i][c] = max(skip, take);
        }

    // Backtrack to find selected items
    vector<int> selected;
    int c = W;
    for (int i = n; i > 0; --i) {
        if (dp[i][c] != dp[i - 1][c]) {
            // Item i-1 was taken
            selected.push_back(i - 1);
            c -= weights[i - 1];
        }
    }
    reverse(selected.begin(), selected.end());
    return selected;
}

// ----- Example usage -----
int main() {
    vector<int> weights = {2, 3, 4, 5};
    vector<int> values = {3, 4, 5, 6};
    int W = 8;

    cout << "Max value: " << knapsack01(W, weights, values) << "\n"; // 10

    auto selected = knapsackReconstruct(W, weights, values);
    cout << "Selected items (0-indexed): ";
    for (int idx : selected) cout << idx << " ";
    cout << "\n"; // 1 3 (items with weight 3 and 5)

    return 0;
}

```

9. Python Implementation

```

# ----- 0/1 Knapsack: 1D Space Optimised -----
def knapsack01(W: int, weights: list[int], values: list[int]) -> int:
    dp = [0] * (W + 1)
    for w, v in zip(weights, values):
        for c in range(W, w - 1, -1): # Descending
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[W]

# ----- 0/1 Knapsack: 2D DP -----
def knapsack01_2d(W: int, weights: list[int], values: list[int]) -> int:
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for c in range(1, W + 1):
            skip = dp[i - 1][c]
            take = dp[i - 1][c - w] + v if w <= c else 0
            dp[i][c] = max(skip, take)
    return dp[n][W]

# ----- 0/1 Knapsack: Reconstruct selected items -----
def knapsack_reconstruct(W: int, weights: list[int], values: list[int]) -> list[int]:
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for c in range(1, W + 1):
            skip = dp[i - 1][c]
            take = dp[i - 1][c - w] + v if w <= c else 0
            dp[i][c] = max(skip, take)

    # Backtrack
    selected = []
    c = W
    for i in range(n, 0, -1):
        if dp[i][c] != dp[i - 1][c]:
            selected.append(i - 1)
            c -= weights[i - 1]
    selected.reverse()
    return selected

# Example
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
W = 8
print(knapsack01(W, weights, values)) # 10
print(knapsack_reconstruct(W, weights, values)) # [1, 3]

```

10. Code Explanation

- **1D DP initialisation:** `vector<int> dp(W + 1, 0)` — all capacities start with value 0 (no items).
- **Outer loop (items):** Each item is considered once — this enforces the "0/1" constraint.
- **Inner loop (descending capacity):** `c` goes from `W` down to `weights[i]`. Descending order ensures `dp[c - w]` hasn't been updated for the current item yet, so the item isn't reused.

- **Transition:** $dp[c] = \max(dp[c], dp[c - w] + v)$ — either skip (keep old $dp[c]$) or take (use $dp[c - w]$ from previous items + current value).
- **Return:** $dp[W]$ is the max value for full capacity.
- **Reconstruction:** The `selected` function backtracks through the 2D table to find which items were taken. If $dp[i][c] \neq dp[i-1][c]$, item $i-1$ was taken. We subtract its weight and continue.
- **2D DP version:** Clearer for understanding but uses $O(nW)$ memory. $dp[i][c]$ = max value using first i items with capacity c .

11. Complexity Analysis

Variant	Time	Space	Notes
2D DP	$O(n \times W)$	$O(n \times W)$	Full table, clear for debugging
1D DP (optimised)	$O(n \times W)$	$O(W)$	Standard for CP and interviews
With reconstruction	$O(n \times W)$	$O(n \times W)$	Needs full table for backtracking
Meet-in-the-middle	$O(2^{(n/2)})$	$O(2^{(n/2)})$	For large W , use when $n \leq 40$

12. Common Patterns

Pattern	Identification	Approach	Example
Standard 0/1 Knapsack	"Max value with weight capacity, each item once"	1D DP descending	Classic
Subset Sum	"Can we achieve exact sum?"	weights = values, check $dp[W] == W$	LeetCode (Partition Equal Subset)
Partition Equal Subset Sum	"Can we split array into two equal sum subsets?"	Target = sum/2, check subset sum	LeetCode 416
Target Sum	"Assign + and - to reach target"	Transform to subset sum	LeetCode 494
Last Stone Weight II	"Min remaining stone weight"	Close to target = sum/2	LeetCode 1049

13. Common Mistakes

- **Ascending capacity loop:** Using `for (c = w; c <= W; c++)` in 1D DP gives unbounded knapsack (items can be used multiple times). Always use **descending** for 0/1.
- **Off-by-one in indices:** $dp[i]$ for item i vs $weight[i-1]$ access.
- **Integer overflow:** Values and counts can be large. Use `long long` if necessary.
- **Not initialising dp correctly:** $dp[0] = 0$ is correct; all unfilled capacities should be 0 (or -INF if you need exact capacity).
- **Forgetting that dp array size must be W+1:** Index W must be accessible.
- **Reconstruction logic error:** Checking $dp[i][c] > dp[i-1][c]$ can fail for equal values. Use $\neq$.
- **Capacity too large for $O(nW)$:** If $W > 10^6$, $O(nW)$ will TLE. Use meet-in-the-middle or DP on value.

14. Edge Cases

Case	Input	Expected	Reason
Empty items	$W=5, items=[]$	0	No items to take
Zero capacity	$W=0$	0	Nothing fits

Case	Input	Expected	Reason
Single item fits	$W=5, w=[3], v=[10]$	10	Take it
Single item doesn't fit	$W=2, w=[3], v=[10]$	0	Can't take
Multiple items, choose best combination	Standard case	DP handles it	Test with small inputs
All items fit	$W=100, \text{sum}(\text{weights})=50$	Sum of all values	Take everything
Large values	Up to 10^9	Use long long	Overflow

15. Variations

Variation	Description	Importance
Subset Sum	Can we achieve exact sum?	High — GFG, many LeetCode problems
Partition Equal Subset Sum	Split into two equal subsets	High — LeetCode 416
Target Sum	+ and - to reach target	High — LeetCode 494
Last Stone Weight II	Min remaining after smashing	Medium — LeetCode 1049
Count of Subsets with Given Sum	Count number of subsets with exact sum	Medium
Minimum Subset Sum Difference	Min	$\text{sum1} - \text{sum2}$
Profit Maximisation	Each job has deadline and profit	Different DP (weighted scheduling)

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Unbounded Knapsack	Items can be taken multiple times	Ascending capacity loop
Fractional Knapsack	Items divisible	Greedy by value/weight ratio
Meet-in-the-Middle	For $n \leq 40$ and large W	Split items into two halves, enumerate
Subset Sum	Special case of 0/1 knapsack	DP with booleans instead of max
Bitmask DP	For very small n (≤ 20)	Enumerate subsets directly

17. Practice Problems

Easy

1. **0/1 Knapsack Problem** — GFG — Standard implementation — Easy/Medium
2. **Subset Sum Problem** — GFG — Check if subset with given sum exists — Medium

Medium

1. **Partition Equal Subset Sum** — LeetCode 416 — Subset sum with target = total/2 — Medium
2. **Target Sum** — LeetCode 494 — Assign + and - to reach target — Medium
3. **Last Stone Weight II** — LeetCode 1049 — Min remaining after smashing — Medium

Hard

1. **Minimum Subset Sum Difference** — GFG — Min $|\text{sum1} - \text{sum2}|$ — Hard
2. **Profit Equalisation** — TopCoder — Complex knapsack variant — Hard

18. Interview Explanation

"The 0/1 Knapsack problem is the foundation of decision DP. For each item, I either take it or skip it, and the decision at each step depends on the remaining capacity. The recurrence is $\text{dp}[c] = \max(\text{dp}[c], \text{dp}[c-w] + v)$, where $\text{dp}[c]$ is the maximum value achievable with capacity c . I use a 1D array and iterate capacity in descending order — this prevents an item from being used more than once. The complexity is $O(n \times W)$ time and $O(W)$ space. I'd mention that when W is very large (e.g., 10^9), we can flip the DP to minimise weight for a given value instead. Common variations include subset sum, partition equal subset sum, and target sum. I'd also demonstrate path reconstruction by backtracking through a 2D table if asked."

19. Revision Notes

- **Key idea:** For each item, $\max(\text{skip}, \text{take with reduced capacity})$.
- **Formula (1D DP):** $\text{dp}[c] = \max(\text{dp}[c], \text{dp}[c - w] + v)$.
- **Descending capacity:** Critical for 0/1 — ascending = unbounded.
- **Initialisation:** $\text{dp}[0..W] = 0$.
- **Complexity:** $O(n \times W)$ time, $O(W)$ space.
- **Traps:** Ascending loop (gives unbounded), overflow, W too large for array.
- **When W is huge:** DP on values instead: $\text{dp}[v] = \min \text{weight to achieve value } v$.

20. Final Cheat Sheet

0/1 KNAPSACK

When to use: Max value with weight capacity, each item at most once

Recurrence: $\text{dp}[c] = \max(\text{dp}[c], \text{dp}[c - w] + v)$ [descending c]

Base cases: $\text{dp}[0..W] = 0$

Time: $O(n \times W)$

Space: $O(W)$

Key code:

```
vector<int> dp(W + 1, 0);
for (int i = 0; i < n; ++i)
    for (int c = W; c >= weights[i]; --c)
        dp[c] = max(dp[c], dp[c - weights[i]] + values[i]);
return dp[W];
```

Edge cases: Empty items, $W=0$, item doesn't fit, overflow

Variations: Subset sum, partition equal, target sum, unbounded

Reconstruction: Backtrack through 2D table ($\text{dp}[i][c] \neq \text{dp}[i-1][c]$)

8. UNBOUNDED KNAPSACK

1. Overview

The Unbounded Knapsack problem is identical to 0/1 Knapsack except that **each item can be taken any number of times** (unlimited supply). Given items with weights $w[i]$ and values $v[i]$, and a capacity W , maximise the total value. The recurrence changes slightly: $dp[c] = \max(dp[c], dp[c - w] + v)$ but now $dp[c - w]$ can already include the current item (since we iterate capacity **ascending**).

2. Intuition

Simple explanation: Since we can reuse items, when considering capacity c , we can take the current item and then look at the best value for capacity $c - w$ — which may already include this item again. This creates an unbounded effect.

Analogy: Imagine you have an unlimited supply of each type of Lego brick. Each brick type has a cost and a point value. You want to fill a box of fixed size to maximise points. For any remaining space, you can always add another brick of the same type.

Step-by-step reasoning:

- For each item, for each capacity from w to W (ascending):
 - You can add this item on top of whatever best combination already fills capacity $c - w$.
 - Since c goes ascending, $dp[c - w]$ might already include the current item.
- This naturally allows unlimited reuse.

Why it works: The ascending loop is the key difference from 0/1. By the time we process capacity c , $dp[c - w]$ has already been updated for the current item, so it can include the item being considered. This effectively allows the item to be used any number of times.

3. When to Use It

- Items can be taken **unlimited** number of times.
- Rod cutting (cut a rod of length n into pieces with given prices).
- Coin change (minimum coins or number of ways).
- Problems with "infinite supply", "unlimited", "any number of times".

Trigger phrases: "Unlimited supply", "infinite", "any number of times", "can be reused", "rod cutting", "coin change (min coins / ways)".

4. When Not to Use It

- When items can be taken **at most once** — use 0/1 knapsack (descending loop).
- When the supply is limited but more than 1 (bounded knapsack) — use multiple 0/1 items or binary splitting.
- When capacity is extremely large and values are small — may still be $O(nW)$ which is too slow.
- When the problem asks for "at most k items" — add a dimension for count.

5. Core Concepts

Concept	Explanation	Why It Matters
State	$dp[c] = \text{max value with capacity } c$	Same as 0/1 knapsack
Transition	$dp[c] = \max(dp[c], dp[c - w] + v)$	BUT with ascending capacity
Ascending loop	$\text{for } c = w..W$	The key difference that allows reuse
Complete knapsack	Another name for unbounded knapsack	Same meaning

Concept	Explanation	Why It Matters
Coin change relation	Coin change (min coins/ways) is a special case of unbounded knapsack	Value = coin value, weight = coin value, goal = make sum

6. Step-by-Step Algorithm

1. Create `dp` array of size `W+1`, initialised to 0.
2. For each item `i` with weight `w` and value `v`:
 - For `c = w` to `W` (ascending):
 - `dp[c] = max(dp[c], dp[c - w] + v)`.
3. Return `dp[W]`.

7. Dry Run

Input:

- `w = [2, 3, 4], v = [3, 4, 7], W = 8`

1D DP Progression (ascending capacity):

Start: `dp = [0, 0, 0, 0, 0, 0, 0, 0, 0]`

Item 1 (w=2, v=3):

c	dp[c-w]	dp[c] new	dp array
2	dp[0]=0	max(0, 0+3)=3	[0,0,3,0,0,0,0,0,0]
3	dp[1]=0	max(0, 0+3)=3	[0,0,3,3,0,0,0,0,0]
4	dp[2]=3	max(0, 3+3)=6	[0,0,3,3,6,0,0,0,0]
5	dp[3]=3	max(0, 3+3)=6	[0,0,3,3,6,6,0,0,0]
6	dp[4]=6	max(0, 6+3)=9	[0,0,3,3,6,6,9,0,0]
7	dp[5]=6	max(0, 6+3)=9	[0,0,3,3,6,6,9,9,0]
8	dp[6]=9	max(0, 9+3)=12	[0,0,3,3,6,6,9,9,12]

Item 2 (w=3, v=4): After item 2: `dp = [0, 0, 3, 4, 6, 7, 9, 10, 12]`

Item 3 (w=4, v=7): After item 3: `dp = [0, 0, 3, 4, 7, 7, 10, 11, 14]`

Answer: 14 (two items of weight 4, value 7 each: 4+4=8 capacity, 7+7=14 value)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Unbounded Knapsack: 1D DP -----
int unboundedKnapsack(int W, vector<int>& weights, vector<int>& values) {
    int n = weights.size();
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
```

```

        // Ascending order allows reuse of the same item
        for (int c = weights[i]; c <= W; ++c) {
            dp[c] = max(dp[c], dp[c - weights[i]] + values[i]);
        }
    }
    return dp[W];
}

// ----- Unbounded Knapsack: 2D DP (for understanding) -----
int unboundedKnapsack2D(int W, vector<int>& weights, vector<int>& values) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; ++i) {
        int w = weights[i - 1];
        int v = values[i - 1];
        for (int c = 1; c <= W; ++c) {
            int skip = dp[i - 1][c];
            int take = (w <= c) ? dp[i][c - w] + v : 0; // Note: dp[i][c-w], not dp[i-1]
            dp[i][c] = max(skip, take);
        }
    }
    return dp[n][W];
}

// ----- Rod Cutting (special case of unbounded knapsack) -----
// Price of length i is price[i-1], rod length = n
int rodCutting(int n, vector<int>& price) {
    // weight = length = index+1, value = price[index]
    vector<int> dp(n + 1, 0);
    for (int len = 1; len <= n; ++len) {
        for (int cut = 1; cut <= len; ++cut) {
            dp[len] = max(dp[len], dp[len - cut] + price[cut - 1]);
        }
    }
    return dp[n];
}

// ----- Example usage -----
int main() {
    vector<int> weights = {2, 3, 4};
    vector<int> values = {3, 4, 7};
    int W = 8;

    cout << "Unbounded max value: " << unboundedKnapsack(W, weights, values) << "\n"; //
14

    // Rod Cutting
    vector<int> price = {1, 5, 8, 9, 10, 17, 17, 20}; // length 1..8
    cout << "Rod cutting max value: " << rodCutting(8, price) << "\n"; // 22
    return 0;
}

```

9. Python Implementation

```
# ----- Unbounded Knapsack: 1D DP -----
def unbounded_knapsack(W: int, weights: list[int], values: list[int]) -> int:
    dp = [0] * (W + 1)
    for w, v in zip(weights, values):
        for c in range(w, W + 1): # Ascending order
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[W]

# ----- Unbounded Knapsack: 2D DP -----
def unbounded_knapsack_2d(W: int, weights: list[int], values: list[int]) -> int:
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for c in range(1, W + 1):
            skip = dp[i - 1][c]
            take = dp[i][c - w] + v if w <= c else 0 # Note: dp[i][c-w], not dp[i-1]
            dp[i][c] = max(skip, take)
    return dp[n][W]

# ----- Rod Cutting -----
def rod_cutting(n: int, price: list[int]) -> int:
    dp = [0] * (n + 1)
    for length in range(1, n + 1):
        for cut in range(1, length + 1):
            dp[length] = max(dp[length], dp[length - cut] + price[cut - 1])
    return dp[n]

# Example
weights = [2, 3, 4]
values = [3, 4, 7]
W = 8
print(unbounded_knapsack(W, weights, values)) # 14

# Rod cutting
price = [1, 5, 8, 9, 10, 17, 17, 20]
print(rod_cutting(8, price)) # 22
```

10. Code Explanation

- **Ascending inner loop:** `for (int c = w; c <= W; ++c)` is the critical difference. When we compute `dp[c]`, `dp[c - w]` may already include the current item, allowing unlimited reuse.
- **2D version difference:** In the 2D version, the recurrence uses `dp[i][c - w]` (same item row) not `dp[i-1][c - w]` (previous item row). This captures the unbounded nature: you can use the current item multiple times.
- **Rod Cutting:** This is the classic example. For each length, you consider all possible cuts. `dp[len] = max(dp[len], dp[len - cut] + price[cut - 1])` means: "either don't cut or cut a piece of length `cut` and add its price to the best for the remaining length."
- **Initialisation:** All `dp` values start at 0 (no items = 0 value). This is correct because we can always choose to not use any items.

11. Complexity Analysis

Variant	Time	Space	Notes
1D DP (optimised)	$O(n \times W)$	$O(W)$	Standard for CP and interviews
2D DP	$O(n \times W)$	$O(n \times W)$	For understanding
Rod Cutting	$O(n^2)$	$O(n)$	Special case: weights = lengths

12. Common Patterns

Pattern	Identification	Approach	Example
Unbounded knapsack	"Infinite supply, maximise value"	$dp[c] = \max(dp[c], dp[c-w] + v)$ ascending	Classic
Rod Cutting	"Cut rod into pieces with given prices"	$dp[len] = \max(dp[len-cut] + price[cut-1])$	GFG
Coin Change (Min Coins)	"Min coins to make amount"	$dp[c] = \min(dp[c], dp[c-coin] + 1)$	LeetCode 322
Coin Change (Ways)	"Number of ways to make amount"	$dp[c] += dp[c - coin]$	LeetCode 518
Integer Break	"Max product of integers summing to n"	$dp[i] = \max(dp[i], j * \max(dp[i-j], i-j))$	LeetCode 343

13. Common Mistakes

- **Using descending loop:** This gives 0/1 knapsack instead of unbounded. Always use **ascending**.
- **Confusing with coin change:** Coin change ways uses **+=** and min coins uses **min** with **+1**. Keep the recurrence straight.
- **Not handling overflow in rod cutting with large n:** Use long long if $n > 60$.
- **Wrong initialisation for min coin change:** $dp[0] = 0$, all others = **INF** (large value).
- **Forgetting that zero capacity means zero value:** All $dp[0]$ should be 0.

14. Edge Cases

Case	Input	Expected	Reason
Empty items	$W=5, items=[]$	0	No items
Zero capacity	$W=0$	0	Nothing fits
Single item	$W=10, w=3, v=5$	15 (3 times)	Take as many as possible
Item weight > W	$W=3, w=5, v=10$	0	Can't use it
All items weight 1	$W=10, w=1, v=vary$	$10 \times \max(v)$	Always pick best value

15. Variations

Variation	Description	Importance
Coin Change (Min Coins)	Minimum number of coins to make amount	High — LeetCode 322

Variation	Description	Importance
Coin Change II (Ways)	Number of combinations to make amount	High — LeetCode 518
Rod Cutting	Maximum profit from cutting rod	High — GFG
Integer Break	Maximum product of summands	Medium — LeetCode 343
Perfect Squares	Min perfect squares summing to n	Medium — LeetCode 279 (min coins pattern)

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
0/1 Knapsack	Same problem, each item at most once	Descending loop for 0/1, ascending for unbounded
Bounded Knapsack	Limited (but not 1) copies	Binary splitting + 0/1 DP
Coin Change	Special case (weight = value)	Same recurrence with min or sum
Greedy (Fractional Knapsack)	Items are divisible	Greedy works only for fractional, not for 0/1 or unbounded

17. Practice Problems

Easy

1. **Rod Cutting** — GFG — Standard unbounded knapsack — Medium
2. **Coin Change (Minimum Coins)** — LeetCode 322 — Min coins to make amount — Medium

Medium

1. **Coin Change II** — LeetCode 518 — Number of ways — Medium
2. **Perfect Squares** — LeetCode 279 — Min perfect squares summing to n — Medium
3. **Combination Sum IV** — LeetCode 377 — Count combos (permutations) — Medium

Hard

1. **Shopping Offers** — LeetCode 638 — Complex unbounded with special offers — Hard
2. **Minimum Cost to Cut a Stick** — LeetCode 1547 — Not unbounded but rod-cutting-like — Hard

18. Interview Explanation

"Unbounded Knapsack differs from 0/1 Knapsack only in that each item can be taken any number of times. The recurrence is the same — $dp[c] = \max(dp[c], dp[c-w] + v)$ — but the capacity loop goes **ascending** instead of descending. This small change means $dp[c-w]$ might already include the current item, allowing unlimited reuse. The complexity remains $O(n \times W)$ time, $O(W)$ space. Classic subproblems are rod cutting, where you cut a rod into pieces of varying lengths with given prices, and coin change, where you make an amount using unlimited coins of given denominations. The key interview insight is recognising when the 'unlimited' constraint changes the loop direction."

19. Revision Notes

- **Key idea:** Same as 0/1 but items can be reused → ascending capacity loop.
- **Formula:** $dp[c] = \max(dp[c], dp[c - w] + v)$ with c from w to W ascending.
- **Difference from 0/1:** 0/1 descends, unbounded ascends.
- **Rod cutting formula:** $dp[len] = \max(dp[len - cut] + price[cut-1])$.
- **Complexity:** $O(n \times W)$ time, $O(W)$ space.
- **Traps:** Using descending loop (gives 0/1), forgetting min vs sum for coin change.

20. Final Cheat Sheet

UNBOUNDED KNAPSACK

When to use: Unlimited supply of each item, maximise value

Recurrence: $dp[c] = \max(dp[c], dp[c - w] + v)$ [ascending c]

Base cases: $dp[0..W] = 0$

Time: $O(n \times W)$

Space: $O(W)$

Key code (diff from 0/1 is loop direction):

```
vector<int> dp(W + 1, 0);
for each item (w, v):
    for (int c = w; c <= W; ++c)    // ASCENDING
        dp[c] = max(dp[c], dp[c - w] + v);
return dp[W];
```

Edge cases: $W=0$, empty items, item weight $> W$

Variations: Rod Cutting, Coin Change (min/ways), Perfect Squares

9. SUBSET SUM

1. Overview

Given a set of integers `nums[]` and a target sum `target`, determine if there exists a subset whose elements sum to exactly `target`. This is a special case of 0/1 Knapsack where `weight = value = nums[i]`. The DP builds a boolean table $dp[i][s] = \text{true}$ if a subset of the first i elements can sum to s .

2. Intuition

Simple explanation: For each number, you either include it in the subset or exclude it. If you include it, the remaining sum decreases by the number's value. If you exclude it, the sum stays the same. The DP tracks which sums are achievable using the first i numbers.

Analogy: You have a set of coins (each can be used at most once) and you want to know if you can pay an exact amount. You check each coin: if you use it, can you make the remaining amount with the remaining coins? If you don't use it, can you make the full amount without it?

Step-by-step reasoning:

1. Start with $dp[0] = \text{true}$ (sum 0 is always achievable — take nothing).
2. For each number `num`:
 - For each sum s from `target` down to `num`:

- If `dp[s - num]` is true (i.e., we can make `s - num` without this number), then `dp[s]` becomes true (by adding this number).

3. Return `dp[target]`.

Why it works: This is exactly 0/1 knapsack but with boolean values. The descending loop ensures each number is used at most once. A subset sum exists for target `t` if either it exists without the current number (`dp[t]` already true), or we can add the current number to a subset that sums to `t - num`.

3. When to Use It

- Determining if a subset with a specific sum exists.
- Partition problems (split array into two equal sum subsets).
- Problems involving "exact sum" with at-most-once usage.
- As a subproblem for harder problems (e.g., min subset sum difference).

Trigger phrases: "Subset sum", "exact sum", "can we achieve sum", "partition into subsets", "is there a subset", "exactly equal to target".

4. When Not to Use It

- When items can be used **multiple times** — use unbounded knapsack pattern.
- When all numbers are **positive** — DP works; if negative numbers exist, need offset or different approach.
- When the target sum is **very large** (10^6+) — $O(n \times \text{target})$ might be too slow; use meet-in-the-middle if n is small (≤ 40).
- When counting **number of subsets** — use count DP instead of boolean (`dp[s] += dp[s - num]`).
- When finding **which subset** (reconstruction) — need 2D table or backtracking.

5. Core Concepts

Concept	Explanation	Why It Matters
State	<code>dp[s]</code> = true if sum <code>s</code> is achievable	1D boolean array
Transition	<code>dp[s] = dp[s] OR dp[s - num]</code>	Include or exclude
Descending loop	<code>for s = target..num</code> descending	Prevents reuse of same element
Base case	<code>dp[0] = true</code>	Empty subset sums to 0
2D table (optional)	<code>dp[i][s]</code> = using first <code>i</code> elements	Useful for reconstruction

6. Step-by-Step Algorithm

1. Let `n = nums.size()`, `total = sum(nums)`.
2. If `target > total`, return false (impossible).
3. Create `dp` of size `target + 1`, all false.
4. `dp[0] = true` (empty subset).
5. For each `num` in `nums`:
 - For `s = target` down to `num`:
 - if (`dp[s - num]`) `dp[s] = true`;
6. Return `dp[target]`.

7. Dry Run

Input: `nums = [3, 2, 7, 1]`, `target = 6`

1D DP progression:

Start: `dp = [T, F, F, F, F, F, F]` (index 0..6)

Num = 3:

s	dp[s-3]	dp[s]	dp array
6	dp[3]=F	F	[T,F,F,F,F,F,F]
5	dp[2]=F	F	[T,F,F,F,F,F,F]
4	dp[1]=F	F	[T,F,F,F,F,F,F]
3	dp[0]=T	T	[T,F,F,T,F,F,F]

Num = 2:

s	dp[s-2]	dp[s]	dp array
6	dp[4]=F	F	[T,F,F,T,F,F,F]
5	dp[3]=T	T	[T,F,F,T,F,T,F]
4	dp[2]=F	F	[T,F,F,T,F,T,F]
3	dp[1]=F	F	(already T)
2	dp[0]=T	T	[T,F,T,T,F,T,F]

Num = 7: 7 > 6, no updates.

Num = 1:

s	dp[s-1]	dp[s]	dp array
6	dp[5]=T	T	[T,F,T,T,F,T,T]
5	dp[4]=F	F	(already T)
4	dp[3]=T	T	[T,F,T,T,T,T,T]
3	dp[2]=T	T	(already T)
2	dp[1]=F	F	(already T)
1	dp[0]=T	T	[T,T,T,T,T,T,T]

Answer: `dp[6] = true` (subset {2, 1, 3} or {7, -1}? Actually {3, 2, 1} = 6)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Subset Sum: 1D boolean DP -----
bool subsetSum(vector<int>& nums, int target) {
    int n = nums.size();
    vector<bool> dp(target + 1, false);
    dp[0] = true; // Empty subset
```

```

    for (int num : nums) {
        // Descending to prevent reuse of same element
        for (int s = target; s >= num; --s) {
            if (dp[s - num])
                dp[s] = true;
        }
    }
    return dp[target];
}

// ----- Subset Sum: With bitset optimisation -----
// Fast for CP when target ≤ 10^5
bool subsetSumBitset(vector<int>& nums, int target) {
    bitset<100001> dp; // Adjust size based on constraints
    dp[0] = 1;

    for (int num : nums) {
        dp |= (dp << num);
    }
    return dp[target];
}

// ----- Subset Sum: Reconstruction -----
vector<int> subsetSumReconstruct(vector<int>& nums, int target) {
    int n = nums.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(target + 1, false));

    for (int i = 0; i <= n; ++i)
        dp[i][0] = true; // sum 0 always achievable

    for (int i = 1; i <= n; ++i) {
        for (int s = 1; s <= target; ++s) {
            dp[i][s] = dp[i - 1][s]; // exclude
            if (!dp[i][s] && s >= nums[i - 1])
                dp[i][s] = dp[i - 1][s - nums[i - 1]]; // include
        }
    }

    // Backtrack
    vector<int> subset;
    if (!dp[n][target]) return subset; // not achievable

    int s = target;
    for (int i = n; i > 0; --i) {
        if (s >= nums[i - 1] && dp[i - 1][s - nums[i - 1]]) {
            subset.push_back(nums[i - 1]);
            s -= nums[i - 1];
        }
    }
    return subset;
}

// ----- Example usage -----
int main() {
    vector<int> nums = {3, 2, 7, 1};
    int target = 6;

```

```

    cout << "Subset sum " << target << " exists: "
        << (subsetSum(nums, target) ? "Yes" : "No") << "\n"; // Yes

    auto subset = subsetSumReconstruct(nums, target);
    cout << "Subset: ";
    for (int x : subset) cout << x << " ";
    cout << "\n"; // 1 2 3 (or similar)

    return 0;
}

```

9. Python Implementation

```

# ----- Subset Sum: 1D boolean DP -----
def subset_sum(nums: list[int], target: int) -> bool:
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for s in range(target, num - 1, -1): # Descending
            if dp[s - num]:
                dp[s] = True
    return dp[target]

# ----- Subset Sum: Reconstruction -----
def subset_sum_reconstruct(nums: list[int], target: int) -> list[int]:
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for s in range(1, target + 1):
            dp[i][s] = dp[i - 1][s] # exclude
            if not dp[i][s] and s >= nums[i - 1]:
                dp[i][s] = dp[i - 1][s - nums[i - 1]] # include

    if not dp[n][target]:
        return []

    # Backtrack
    subset = []
    s = target
    for i in range(n, 0, -1):
        if s >= nums[i - 1] and dp[i - 1][s - nums[i - 1]]:
            subset.append(nums[i - 1])
            s -= nums[i - 1]
    return subset

# Example
nums = [3, 2, 7, 1]
target = 6
print(subset_sum(nums, target)) # True
print(subset_sum_reconstruct(nums, target)) # [1, 2, 3]

```

10. Code Explanation

- **1D boolean DP:** `dp[s]` tracks if sum `s` is achievable. `dp[0] = true` because empty subset sums to 0.
- **Descending loop:** `for (int s = target; s >= num; --s)` — this ensures each number is used at most once, just like 0/1 knapsack.
- **Optimisation:** Early exit — if `dp[target]` becomes true, we could break early, but it's not always worth the check.
- **Bitset version:** `dp |= (dp << num)` shifts all bits by `num` (adding `num` to every existing sum) and ORs with the current set. This is highly efficient in C++ because `bitset` operations are word-level ($\sim O(\text{target}/\text{word_size})$) and very fast in practice.
- **Reconstruction:** The 2D table tracks which sums are achievable with first `i` items. Backtracking goes from `dp[n][target]` backwards: if `dp[i-1][s]` is true (achievable without item `i-1`), the item was NOT taken. Otherwise, it was taken.

11. Complexity Analysis

Variant	Time	Space	Notes
1D boolean DP	$O(n \times \text{target})$	$O(\text{target})$	Standard
2D boolean DP	$O(n \times \text{target})$	$O(n \times \text{target})$	For reconstruction
Bitset	$O(n \times \text{target} / \text{word_size})$	$O(\text{target})$	Fast in C++; $\text{target} \leq \sim 10^5$
Meet-in-the-middle	$O(2^{(n/2)})$	$O(2^{(n/2)})$	For $n \leq 40$ and large target

12. Common Patterns

Pattern	Identification	Approach	Example
Exact subset sum	"Is there a subset summing to target?"	<code>dp[s] = dp[s] OR dp[s - num]</code>	Classic
Partition equal subset sum	"Split array into two equal sum subsets"	<code>target = total/2</code> , check subset sum	LeetCode 416
Target Sum	"Assign + and - to reach target"	Transform to subset sum	LeetCode 494
Minimum subset sum difference	"Min	<code>sum1 - sum2</code>	"
Count subsets with sum	"How many subsets sum to target?"	<code>dp[s] += dp[s - num]</code>	Variation

13. Common Mistakes

- **Using ascending loop:** This would allow using the same element multiple times (unbounded).
- **Not checking if target > total sum:** Early return false saves time.
- **Forgetting `dp[0] = true`:** Without this, no sums can be made.
- **Using int for dp:** Boolean (`bool` or `vector<bool>`) is more memory-efficient.
- **Overflow with bitset:** Bitset size must be known at compile time. Use `vector<bool>` for dynamic sizes.
- **Reconstruction with 1D DP:** Can't reconstruct from 1D DP alone; need 2D table.

14. Edge Cases

Case	Input	Expected	Reason
target = 0	any nums	true	Empty subset
target > total	nums=[1,2], target=10	false	Impossible
target = total	nums=[1,2,3], target=6	true	All elements
Single element matches	nums=[5], target=5	true	Exactly that element
All zeros	nums=[0,0,0], target=0	true	Use empty subset or zeros
Large numbers	nums=[10 ⁹], target=10 ⁹	true	Use long long for sum

15. Variations

Variation	Description	Importance
Partition Equal Subset Sum	Can we split into two equal sum subsets?	High — LeetCode 416
Target Sum	Count ways to assign + and - to reach target	High — LeetCode 494
Minimum Subset Sum Difference	Min absolute difference between two subset sums	High — GFG
Subset Sum Count	Count number of subsets with given sum	Medium
Subset Sum with negatives	nums can be negative	Hard — requires offsetting indices

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
0/1 Knapsack	Subset sum = 0/1 knapsack where weight = value = num	Use subset sum for boolean existence
Partition Equal Subset	Direct application of subset sum	Check if target = total/2 is achievable
Meet-in-the-Middle	Alternative for large target and small n (≤ 40)	Use when $O(n \times \text{target})$ is too large
Two Sum / Three Sum	Different: find two/three elements summing to target	Use hashing/two-pointer instead of DP

17. Practice Problems

Easy

1. **Subset Sum Problem** — GFG — Standard boolean — Medium
2. **Partition Equal Subset Sum** — LeetCode 416 — Check if split possible — Medium

Medium

1. **Target Sum** — LeetCode 494 — Count ways to reach target — Medium
2. **Minimum Subset Sum Difference** — GFG — Min $|\text{sum1} - \text{sum2}|$ — Hard

Hard

1. **Last Stone Weight II** — LeetCode 1049 — Subset sum minimisation — Medium
2. **Number of Subsequences That Satisfy the Given Sum Condition** — LeetCode 1498 — Two-pointer + counting — Medium

18. Interview Explanation

"Subset Sum is a special case of 0/1 Knapsack where weight equals value. I use a 1D boolean array `dp` where `dp[s]` indicates whether sum `s` is achievable. The base case is `dp[0] = true` — the empty subset. For each number, I iterate capacity descending from target down to the number, setting `dp[s] = true` if `dp[s - num]` is true. The descending order ensures each number is used at most once. Time is $O(n \times \text{target})$ and space is $O(\text{target})$. If asked to reconstruct, I use a 2D table and backtrack. If target is very large but `n` is small (≤ 40), I'd use meet-in-the-middle. A classic application is the Partition Equal Subset Sum problem where I check if `target = total_sum/2` is achievable."

19. Revision Notes

- **Key idea:** Can we sum to target using each element at most once?
- **Formula:** `dp[s] = dp[s] OR dp[s - num]` with descending `s`.
- **Base case:** `dp[0] = true`.
- **Template:** `dp[0] = true; for num: for s = target down to num: if dp[s-num]: dp[s] = true;`
- **Complexity:** $O(n \times \text{target})$ time, $O(\text{target})$ space.
- **Traps:** Ascending loop gives unbounded reuse; `target > total sum` → false.
- **Bitset optimisation:** `bitset<MAX> dp; dp[0] = 1; for num: dp |= (dp << num);`

20. Final Cheat Sheet

SUBSET SUM

When to use: Can we achieve exact sum using each element at most once?

Recurrence: `dp[s] = dp[s] OR dp[s - num]` [descending `s`]

Base cases: `dp[0] = true`

Time: $O(n \times \text{target})$

Space: $O(\text{target})$

Key code:

```
vector<bool> dp(target + 1, false);
dp[0] = true;
for (int num : nums)
    for (int s = target; s >= num; --s)
        if (dp[s - num]) dp[s] = true;
return dp[target];
```

Edge cases: `target = 0` → true; `target > total` → false

Variations: Partition equal subset, target sum, min diff, count

Bitset trick: `bitset<MAX> dp; dp[0]=1; for(num) dp |= (dp << num);`

10. PARTITION EQUAL SUBSET SUM

1. Overview

Given an integer array `nums`, return `true` if you can partition the array into **two subsets** such that the sum of elements in both subsets is **equal**. This is a direct application of the Subset Sum problem: if the total sum is `S`, we need to find a subset that sums to exactly `S/2`. If `S` is odd, partition is impossible.

2. Intuition

Simple explanation: If the total sum is even, check if there is a subset whose sum is exactly half the total. The remaining elements automatically form the other half (with equal sum). This is exactly the Subset Sum problem with target = total/2.

Analogy: You have a pile of coins and want to split them into two piles of equal value. You don't need to split each coin — just find one group that sums to half the total. The rest is the other group.

Step-by-step reasoning:

1. Compute total sum `S`.
2. If `S` is odd, return false (can't split equally).
3. Target = `S / 2`.
4. Use Subset Sum DP to check if target is achievable.
5. Return result.

Why it works: If a subset with sum `S/2` exists, the remaining elements (complement subset) also sum to `S - S/2 = S/2`. So both subsets have equal sum. The problem reduces to Subset Sum.

3. When to Use It

- "Can you split the array into two equal sum subsets?"
- Any problem phrased as "partition into two subsets with equal sum".
- As a warm-up to Subset Sum in interviews.

Trigger phrases: "Partition", "equal sum", "split into two subsets", "can be divided into two groups", "same sum".

4. When Not to Use It

- When partitioning into **more than 2 subsets** (e.g., k-partition) — different DP with bitmask or backtracking.
- When total sum is **odd** — immediate false.
- When elements are **not integers** (floats) — DP on floats doesn't work; use meet-in-the-middle or backtracking.
- When the array is **very large** ($n > 200$ and $\text{sum} > 10^5$) — $O(n \times \text{sum}/2)$ may be slow; need optimisation.

5. Core Concepts

Concept	Explanation	Why It Matters
Total sum	Sum of all elements	Must be even for equal partition
Target	<code>total / 2</code>	The sum we need to achieve
Reduction	Partition Equal Subset Sum = Subset Sum with target = total/2	Why DP works
Early exit	Odd sum → false	Optimisation before DP
DP on booleans	Same as Subset Sum 1D boolean DP	Reuse existing knowledge

6. Step-by-Step Algorithm

1. Compute `total = sum(nums)`.
2. If `total % 2 == 1`, return false.
3. `target = total / 2`.
4. Create `dp` of size `target + 1`, all false.
5. `dp[0] = true`.
6. For each `num` in `nums`:
 - For `s = target` down to `num`:
 - If `dp[s - num]`, set `dp[s] = true`.
 - (Optional) If `dp[target]` is true, break early.
7. Return `dp[target]`.

7. Dry Run

Input: `nums = [1, 5, 11, 5]`

- Total = 22, target = 11

1D DP progression:

Start: `dp = [T, F, F, F, F, F, F, F, F, F, F, F, F]` (indices 0..11)

Num = 1: `dp = [T, T, F, F, F, F, F, F, F, F, F, F, F]`

Num = 5: `s=11..5`: `dp[5]=T (dp[0])`, `dp[6]=T (dp[1])`, `dp[7]=dp[8]=dp[9]=dp[10]=dp[11]=F` `dp = [T, T, F, F, F, T, T, F, F, F, F, F, F]`

Num = 11: `s=11`: `dp[0]=T → dp[11]=T` `dp = [T, T, F, F, F, T, T, F, F, F, F, T, T]`

Num = 5: `s=11..5`: Already T for many; `dp[10]=T (dp[5])`, `dp[11]` stays T `dp = [T, T, F, F, F, T, T, F, F, F, T, T, T]`

Answer: `dp[11] = true` X (Wait, let me re-check)

Actually let me redo carefully:

Num = 1: `s=11..1`: `dp[1]=T (dp[0])` `dp = [T, T, F, F, F, F, F, F, F, F, F, F, F]`

Num = 5 (first): `s=11`: `dp[6]=?` `dp[6]=F → no` `s=10`: `dp[5]=F → no ...` `s=6`: `dp[1]=T → dp[6]=T` `s=5`: `dp[0]=T → dp[5]=T`
`dp = [T, T, F, F, F, T, T, F, F, F, F, F, F]`

Num = 11: `s=11`: `dp[0]=T → dp[11]=T` `dp = [T, T, F, F, F, T, T, F, F, F, T, T, T]`

Num = 5 (second): `s=11`: `dp[6]=T → dp[11]=T (already)` `s=10`: `dp[5]=T → dp[10]=T` `s=9`: `dp[4]=F → F` `s=8`: `dp[3]=F → F`
`s=7`: `dp[2]=F → F` `s=6`: `dp[1]=T → dp[6]=T (already)` `s=5`: `dp[0]=T → dp[5]=T (already)` `dp = [T, T, F, F, F, T, T, F, F, F, T, T, T]`

Answer: `dp[11] = true` ✓ (subset {11} or {1, 5, 5} = 11)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Partition Equal Subset Sum -----
// LeetCode 416
```

```

bool canPartition(vector<int>& nums) {
    int total = accumulate(nums.begin(), nums.end(), 0);

    // Odd sum cannot be split into two equal integers
    if (total % 2 == 1) return false;

    int target = total / 2;
    vector<bool> dp(target + 1, false);
    dp[0] = true;

    for (int num : nums) {
        // Descending to prevent reuse of same element
        for (int s = target; s >= num; --s) {
            if (dp[s - num])
                dp[s] = true;
        }
        // Early exit: if target is reachable, we're done
        if (dp[target]) return true;
    }

    return dp[target];
}

// ----- Partition Equal Subset Sum: With pruning -----
// If any element > target, false; also sort for better early exit
bool canPartitionPruned(vector<int>& nums) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    if (total % 2 == 1) return false;

    int target = total / 2;

    // Quick check: if any element > target, it can't be in any subset
    for (int num : nums)
        if (num > target) return false;

    // Sort to process larger numbers first (better pruning)
    sort(nums.begin(), nums.end(), greater<int>());

    vector<bool> dp(target + 1, false);
    dp[0] = true;

    for (int num : nums) {
        for (int s = target; s >= num; --s) {
            if (dp[s - num])
                dp[s] = true;
        }
        if (dp[target]) return true;
    }

    return dp[target];
}

// ----- Partition Equal Subset Sum: Recursive with memo -----
// Top-down alternative
bool dfs(vector<int>& nums, int i, int target, vector<vector<int>>& memo) {
    if (target == 0) return true;
    if (i >= nums.size() || target < 0) return false;
    if (memo[i][target] != -1) return memo[i][target];

```

```

    bool include = dfs(nums, i + 1, target - nums[i], memo);
    bool exclude = dfs(nums, i + 1, target, memo);

    return memo[i][target] = (include || exclude);
}

bool canPartitionTopDown(vector<int>& nums) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    if (total % 2 == 1) return false;

    int target = total / 2;
    vector<vector<int>> memo(nums.size(), vector<int>(target + 1, -1));
    return dfs(nums, 0, target, memo);
}

// ----- Example usage -----
int main() {
    vector<int> nums = {1, 5, 11, 5};
    cout << "Can partition: " << (canPartition(nums) ? "Yes" : "No") << "\n"; // Yes

    vector<int> nums2 = {1, 2, 3, 5};
    cout << "Can partition: " << (canPartition(nums2) ? "Yes" : "No") << "\n"; // No

    return 0;
}

```

9. Python Implementation

```

# ----- Partition Equal Subset Sum -----
def can_partition(nums: list[int]) -> bool:
    total = sum(nums)
    if total % 2 == 1:
        return False

    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True

    for num in nums:
        for s in range(target, num - 1, -1):
            if dp[s - num]:
                dp[s] = True
        if dp[target]:
            return True

    return dp[target]

# ----- Partition Equal Subset Sum: Top-down -----
def can_partition_top_down(nums: list[int]) -> bool:
    total = sum(nums)
    if total % 2 == 1:
        return False

    target = total // 2
    memo = {}

```

```
def dfs(i: int, t: int) -> bool:
    if t == 0:
        return True
    if i >= len(nums) or t < 0:
        return False
    if (i, t) in memo:
        return memo[(i, t)]

    include = dfs(i + 1, t - nums[i])
    exclude = dfs(i + 1, t)
    memo[(i, t)] = include or exclude
    return memo[(i, t)]

return dfs(0, target)

# Example
print(can_partition([1, 5, 11, 5])) # True
print(can_partition([1, 2, 3, 5])) # False
```

10. Code Explanation

- **Odd sum check:** `if (total % 2 == 1) return false;` — immediate rejection. Equal sum means total must be even.
- **Target:** `target = total / 2` — the sum we need one subset to achieve.
- **DP initialisation:** `dp[0] = true` — we can always achieve sum 0 (take no elements from the subset).
- **Descending loop:** Same as 0/1 knapsack — each element used at most once.
- **Early exit:** `if (dp[target]) return true;` — once the target is achievable, no need to process remaining elements.
- **Pruning:** If any element is larger than target, it can't be in any valid subset with sum = target (since all numbers are positive). Also sorting descending helps reach target faster.
- **Top-down version:** `dfs(i, target)` checks if we can achieve `target` using elements from index `i` onward. Base case: `target == 0` → true. Recursive: include or exclude current element.

11. Complexity Analysis

Variant	Time	Space	Notes
Bottom-up 1D	$O(n \times \text{target})$	$O(\text{target})$	Standard
Top-down memo	$O(n \times \text{target})$	$O(n \times \text{target})$	Recursion stack
With pruning	$O(n \times \text{target})$	$O(\text{target})$	Better constants

12. Common Patterns

Pattern	Identification	Approach	Example
Equal partition	"Split into two equal sum subsets"	<code>target = total/2</code> , subset sum DP	LeetCode 416
K-partition	"Split into k equal sum subsets"	Backtracking / bitmask DP	LeetCode 698
Target Sum	"Assign + and - to reach target"	Transform: count subsets	LeetCode 494
Min subset sum diff	"Min	<code>sum1 - sum2</code>	"

13. Common Mistakes

- **Forgetting odd sum check:** Without it, DP runs on non-integer target (if total is odd, $\text{total}/2$ truncates).
- **Using `int` for total when sum may be large:** Use `long long` if values can be up to 10^9 and n up to 200.
- **Ascending loop:** Leads to unbounded usage — each element can be used multiple times.
- **Not breaking early:** Processing all elements even after target is found wastes time.
- **Assuming all numbers are positive:** If zeros exist, they don't affect sum but can cause issues in some variants.
- **Memory issues with large target:** target could be up to 10^5 or more — `vector<bool>` is space-efficient (bits), but still $O(\text{target})$.

14. Edge Cases

Case	Input	Expected	Reason
Odd total	<code>[1, 2, 2]</code> sum=5	false	Can't split equally
Two elements equal	<code>[5, 5]</code>	true	Each in own subset
All elements zero	<code>[0, 0, 0]</code>	true	Both subsets sum to 0
Large element > half	<code>[10, 1, 2, 3]</code> sum=16	false	$10 > 8$, can't fit
Single element	<code>[5]</code> sum=5	false	Need at least 2 subsets
Already partitioned	<code>[1, 2, 3, 4, 5, 6]</code>	true	$1+2+4+5=6+6?$ sum=21, odd $\rightarrow$ false

15. Variations

Variation	Description	Importance
K Partition Equal Subset Sum	Split into k equal sum subsets	Hard — LeetCode 698
Partition to K Equal Sum Subsets	Same as above	Hard — LeetCode 698
Minimum Subset Sum Difference	Min	sum1 - sum2
Target Sum	Count ways to assign signs to reach target	High — LeetCode 494

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Subset Sum	Direct reduction	Use for target = total/2
0/1 Knapsack	Generalisation	Use for weighted value problems
Backtracking	Alternative for small n	Use when $n \leq 20$
Bitmask DP	For k -partition ($k \geq 3$)	Use with bitmask for state of taken elements

17. Practice Problems

Easy

1. **Partition Equal Subset Sum** — LeetCode 416 — Standard — Medium
2. **Subset Sum Problem** — GFG — Same DP — Medium

Medium

1. **Target Sum** — LeetCode 494 — Sign assignment counting — Medium
2. **Minimum Subset Sum Difference** — GFG — Find closest to half — Hard

Hard

1. **Partition to K Equal Sum Subsets** — LeetCode 698 — Generalised — Hard
2. **Can I Win** — LeetCode 464 — Game with subset-like mechanics — Medium

18. Interview Explanation

"Partition Equal Subset Sum is a direct application of the Subset Sum problem. First, I check if the total sum is even — if not, return false immediately. Then I run Subset Sum DP with target = total/2, using a 1D boolean array and descending capacity loop to ensure each element is used at most once. Time is $O(n \times \text{total}/2)$ and space is $O(\text{total}/2)$. I can add an early exit once the target is found, and prune elements larger than the target. The key insight is that finding one subset with sum = total/2 automatically gives us the other subset as the complement."

19. Revision Notes

- **Key idea:** If total is even and subset sum to total/2 exists, partition is possible.
- **Formula:** Same as subset sum: $dp[s] = dp[s] \text{ OR } dp[s - \text{num}]$ with descending s.
- **Early check:** `if (total % 2 == 1) return false;`
- **Template:** `target = total/2; dp[0]=true; for num: for s=target..num: if dp[s-num]: dp[s]=true;`
- **Complexity:** $O(n \times \text{total}/2)$ time, $O(\text{total}/2)$ space.
- **Traps:** Odd sum, element > target, ascending loop, forgetting dp[0].

20. Final Cheat Sheet

PARTITION EQUAL SUBSET SUM

When to use: Split array into two equal-sum subsets
 Reduction: Subset Sum with target = total_sum / 2
 Key check: `if (total % 2 == 1) return false`
 Recurrence: `dp[s] = dp[s] OR dp[s - num]` [descending s]
 Time: $O(n \times \text{total}/2)$
 Space: $O(\text{total}/2)$

Key code:

```
int total = accumulate(nums.begin(), nums.end(), 0);
if (total % 2 == 1) return false;
int target = total / 2;
vector<bool> dp(target + 1, false);
dp[0] = true;
for (int num : nums) {
    for (int s = target; s >= num; --s)
        if (dp[s - num]) dp[s] = true;
    if (dp[target]) return true;
}
return dp[target];
```

Edge cases: Odd total → false; element > target → can prune
 Variations: K-partition, target sum, min subset diff

11. COIN CHANGE

1. Overview

Coin Change problems come in two flavours:

1. **Minimum Coins** (LeetCode 322): Given coins of different denominations and an amount, find the **minimum number of coins** needed to make that amount. If impossible, return -1.
2. **Coin Change II (Ways)** (LeetCode 518): Count the **number of combinations** that make up that amount.

Both are Unbounded Knapsack problems (coins can be used unlimited times). Minimum Coins uses `min` with `+1`; Ways uses `+=`.

2. Intuition

Simple explanation (Min Coins): For each coin, you decide to use it once and then find the minimum coins needed for the remaining amount. Since coins are unlimited, you keep trying the same coin.

Analogy: You have unlimited ₹1, ₹2, and ₹5 coins. To make ₹11, you try: use one ₹5 (then need ₹6, which needs 3 coins → total 4), or use one ₹2 (then need ₹9 → 5 coins), etc. The minimum is 3 coins (₹5 + ₹5 + ₹1).

Simple explanation (Ways): For each coin, the number of ways to make amount `a` is the ways without using this coin plus the ways using this coin (i.e., ways to make `a - coin`).

Step-by-step reasoning (Min Coins):

1. `dp[a]` = minimum coins to make amount `a`.
2. `dp[0] = 0` (0 coins needed for 0 amount).
3. For each coin, for each amount `a` from coin to target:
 - `dp[a] = min(dp[a], dp[a - coin] + 1)`.
4. If `dp[target]` is INF, return -1.

Why it works: This is unbounded knapsack because we iterate amounts ascending (allowing the same coin to be used multiple times). The `min` ensures we track the optimal number of coins.

3. When to Use It

- "Minimum number of coins to make an amount" (Min Coins).
- "Number of ways / combinations to make an amount" (Coin Change II).
- Any problem with unlimited supplies and a target sum.

Trigger phrases: "Coin change", "minimum coins", "number of ways to make", "unlimited supply", "make a sum using denominations".

4. When Not to Use It

- When coins can be used **at most once** (0/1) — use subset sum DP.
- When the amount is **very large** (10^6+) and number of coins is small — $O(n \times \text{amount})$ may be slow; consider BFS (since coin change is like shortest path in a graph).
- When finding **which coins** (not just count) — need parent pointers.
- When the coin denominations are **weird** with large gaps — DP still works but may have many unreachable amounts.

5. Core Concepts

Concept	Explanation	Why It Matters
State	$dp[a] = \text{min coins (or number of ways) for amount } a$	1D DP array
Transition (min)	$dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$	Use one coin of this denomination
Transition (ways)	$dp[a] += dp[a - \text{coin}]$	Add ways using this coin
Unbounded nature	Ascending loop over amounts	Coins can be reused
Initialisation (min)	$dp[0] = 0$, all others = INF	Base case for min coins
Initialisation (ways)	$dp[0] = 1$, all others = 0	Base case for counting

6. Step-by-Step Algorithm

Minimum Coins:

1. Create `dp` of size `amount + 1`, initialise to `amount + 1` (or a large value).
2. $dp[0] = 0$.
3. For each `coin` in `coins`:
 - For $a = \text{coin}$ to `amount` (ascending):
 - $dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$.
4. Return $dp[\text{amount}]$ if it's not INF, else -1.

Coin Change II (Ways):

1. Create `dp` of size `amount + 1`, all 0.
2. $dp[0] = 1$ (1 way to make amount 0: take nothing).
3. For each `coin` in `coins`:
 - For $a = \text{coin}$ to `amount` (ascending):
 - $dp[a] += dp[a - \text{coin}]$.
4. Return $dp[\text{amount}]$.

7. Dry Run

Minimum Coins: Input: `coins = [1, 2, 5]`, `amount = 11`

a	initial	after coin=1	after coin=2	after coin=5
0	0	0	0	0
1	INF	1	1	1
2	INF	2	1	1
3	INF	3	2	2
4	INF	4	2	2
5	INF	5	3	1
6	INF	6	3	2
7	INF	7	4	2
8	INF	8	4	3
9	INF	9	5	3

a	initial	after coin=1	after coin=2	after coin=5
10	INF	10	5	2
11	INF	11	6	3

Answer: 3 (5 + 5 + 1)

Coin Change II (Ways): Input: `coins = [1, 2, 5]`, `amount = 5`

a	initial	after coin=1	after coin=2	after coin=5
0	1	1	1	1
1	0	1	1	1
2	0	1	2	2
3	0	1	2	2
4	0	1	3	3
5	0	1	3	4

Answer: 4 ways: (5), (2+2+1), (2+1+1+1), (1+1+1+1+1)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- Minimum Coins -----
// LeetCode 322
int coinChangeMin(vector<int>& coins, int amount) {
    const int INF = amount + 1; // Larger than any possible answer
    vector<int> dp(amount + 1, INF);
    dp[0] = 0;

    for (int coin : coins) {
        for (int a = coin; a <= amount; ++a) {
            dp[a] = min(dp[a], dp[a - coin] + 1);
        }
    }

    return (dp[amount] == INF) ? -1 : dp[amount];
}

// ----- Coin Change II (Number of Ways) -----
// LeetCode 518
int coinChangeWays(vector<int>& coins, int amount) {
    vector<unsigned long long> dp(amount + 1, 0);
    dp[0] = 1; // One way: use no coins

    for (int coin : coins) {
        for (int a = coin; a <= amount; ++a) {
            dp[a] += dp[a - coin];
        }
    }
}
```

```

    return dp[amount];
}

// ----- Minimum Coins: BFS (alternative for large amounts) -----
// Treat as shortest path in a graph where nodes are amounts
int coinChangeBFS(vector<int>& coins, int amount) {
    if (amount == 0) return 0;

    vector<int> dist(amount + 1, -1);
    queue<int> q;
    dist[0] = 0;
    q.push(0);

    while (!q.empty()) {
        int cur = q.front(); q.pop();
        for (int coin : coins) {
            int next = cur + coin;
            if (next <= amount && dist[next] == -1) {
                dist[next] = dist[cur] + 1;
                if (next == amount) return dist[next];
                q.push(next);
            }
        }
    }
    return -1;
}

// ----- Minimum Coins: Reconstruction -----
vector<int> coinChangeReconstruct(vector<int>& coins, int amount) {
    const int INF = amount + 1;
    vector<int> dp(amount + 1, INF);
    vector<int> usedCoin(amount + 1, -1); // First coin used to reach this amount
    dp[0] = 0;

    for (int coin : coins) {
        for (int a = coin; a <= amount; ++a) {
            if (dp[a - coin] + 1 < dp[a]) {
                dp[a] = dp[a - coin] + 1;
                usedCoin[a] = coin;
            }
        }
    }

    vector<int> result;
    if (dp[amount] == INF) return result;

    int cur = amount;
    while (cur > 0) {
        result.push_back(usedCoin[cur]);
        cur -= usedCoin[cur];
    }
    return result;
}

// ----- Example usage -----
int main() {
    vector<int> coins = {1, 2, 5};

```

```

    int amount = 11;

    cout << "Min coins: " << coinChangeMin(coins, amount) << "\n"; // 3
    cout << "Number of ways: " << coinChangeWays(coins, 5) << "\n"; // 4

    auto used = coinChangeReconstruct(coins, amount);
    cout << "Coins used: ";
    for (int c : used) cout << c << " ";
    cout << "\n"; // 5 5 1

    return 0;
}

```

9. Python Implementation

```

# ----- Minimum Coins -----
def coin_change_min(coins: list[int], amount: int) -> int:
    INF = amount + 1
    dp = [INF] * (amount + 1)
    dp[0] = 0

    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] = min(dp[a], dp[a - coin] + 1)

    return -1 if dp[amount] == INF else dp[amount]

# ----- Coin Change II (Ways) -----
def coin_change_ways(coins: list[int], amount: int) -> int:
    dp = [0] * (amount + 1)
    dp[0] = 1

    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] += dp[a - coin]

    return dp[amount]

# ----- Minimum Coins: BFS -----
from collections import deque
def coin_change_bfs(coins: list[int], amount: int) -> int:
    if amount == 0:
        return 0

    dist = [-1] * (amount + 1)
    q = deque([0])
    dist[0] = 0

    while q:
        cur = q.popleft()
        for coin in coins:
            nxt = cur + coin
            if nxt <= amount and dist[nxt] == -1:
                dist[nxt] = dist[cur] + 1
                if nxt == amount:
                    return dist[nxt]
            q.append(nxt)

```

```

        q.append(nxt)

    return -1

# ----- Reconstruction -----
def coin_change_reconstruct(coins: list[int], amount: int) -> list[int]:
    INF = amount + 1
    dp = [INF] * (amount + 1)
    used = [-1] * (amount + 1)
    dp[0] = 0

    for coin in coins:
        for a in range(coin, amount + 1):
            if dp[a - coin] + 1 < dp[a]:
                dp[a] = dp[a - coin] + 1
                used[a] = coin

    if dp[amount] == INF:
        return []

    result = []
    cur = amount
    while cur > 0:
        result.append(used[cur])
        cur -= used[cur]
    return result

# Example
coins = [1, 2, 5]
print(coin_change_min(coins, 11)) # 3
print(coin_change_ways(coins, 5)) # 4
print(coin_change_reconstruct(coins, 11)) # [5, 5, 1]

```

10. Code Explanation

- **Min Coins — Initialisation:** $dp[0] = 0$, all others = $amount + 1$ (a sentinel larger than any possible answer, since the max coins needed is $amount$ using coin 1).
- **Min Coins — Transition:** $dp[a] = \min(dp[a], dp[a - coin] + 1)$. This means: either don't use this coin (keep $dp[a]$ as is), or use one coin of this denomination and add 1 to the min coins needed for the remaining amount.
- **Min Coins — Return:** If $dp[amount]$ is still INF, return -1.
- **Ways — Initialisation:** $dp[0] = 1$ — there is exactly 1 way to make amount 0 (choose nothing).
- **Ways — Transition:** $dp[a] += dp[a - coin]$. This adds the number of ways that use this coin to the existing ways (which don't use this coin).
- **Ascending loop:** Both versions iterate a from $coin$ to $amount$ ascending. This is the unbounded knapsack pattern — the same coin can be used multiple times.
- **BFS version:** Treat amounts as nodes, coins as edges. BFS finds the shortest path (minimum coins). This can be faster when amount is large and the coin graph is sparse (few coins).
- **Reconstruction:** The `usedCoin` array stores which coin was used to achieve each amount. We backtrack from $amount$ down to 0.

11. Complexity Analysis

Variant	Time	Space	Notes
Min Coins DP	$O(n \times \text{amount})$	$O(\text{amount})$	Standard
Ways DP	$O(n \times \text{amount})$	$O(\text{amount})$	Standard
BFS (min coins)	$O(\text{amount} \times n)$ worst	$O(\text{amount})$	Can be faster in practice
Reconstruction	$O(n \times \text{amount})$	$O(\text{amount})$	Same as DP + parent array

12. Common Patterns

Pattern	Identification	Approach	Example
Min Coins	"Minimum number of coins"	$\text{dp}[a] = \min(\text{dp}[a], \text{dp}[a - \text{coin}] + 1)$	LeetCode 322
Coin Change II (Ways)	"Number of combinations"	$\text{dp}[a] += \text{dp}[a - \text{coin}]$	LeetCode 518
Perfect Squares	"Min perfect squares summing to n"	Same as min coins	LeetCode 279
Combination Sum IV	"Number of permutations (order matters)"	$\text{dp}[a] += \text{dp}[a - \text{num}]$ (outside loop = permutations)	LeetCode 377
Minimum Cost for Tickets	"Min cost to cover days"	Different DP but similar min pattern	LeetCode 983

13. Common Mistakes

- **Confusing min and ways recurrence:** Min uses $\min(\text{dp}[a], \text{dp}[a - \text{coin}] + 1)$. Ways uses $\text{dp}[a] += \text{dp}[a - \text{coin}]$.
- **Wrong loop order for ways:** Outer loop over coins gives **combinations** (order doesn't matter). Outer loop over amount gives **permutations** (different orders count separately).
- **Not checking for -1 (impossible):** In min coins, if $\text{dp}[\text{amount}]$ is still INF, return -1.
- **Overflow in ways:** The number of ways can be huge. LeetCode 518 guarantees 32-bit int, but some problems may need `long long` or modulo.
- **Ascending vs descending:** Ascending for unbounded (coin change). Descending for 0/1 (subset sum). Getting this wrong is a common mistake.
- **Initialisation for min coins:** Using `INT_MAX` can cause overflow when adding 1. Use $\text{amount} + 1$ or `INF` and check before adding.

14. Edge Cases

Case	Input	Expected	Reason
amount = 0	any coins	0 (min), 1 (ways)	Base case
No coins	<code>coins=[], amount=5</code>	-1 (min), 0 (ways)	Impossible
Single coin matches	<code>coins=[5], amount=5</code>	1 (min), 1 (ways)	Exactly that coin
Single coin doesn't match	<code>coins=[3], amount=5</code>	-1 (min), 0 (ways)	Not divisible
Large amount	<code>coins=[1], amount=10^4</code>	10000 (min), 1 (ways)	Large but works

15. Variations

Variation	Description	Importance
Combination Sum IV	Count permutations (order matters)	Medium — LeetCode 377
Perfect Squares	Min perfect squares summing to n	Medium — LeetCode 279
Minimum Cost for Tickets	Min cost covering travel days	Medium — LeetCode 983
Coin Change with limited coins	Bounded knapsack variant	Medium — harder than unbounded
Coin Change with negative coins	Very rare in CP	Avoid — different theory needed

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Unbounded Knapsack	Generalisation of coin change	Use coin change for min/ways specifically
BFS	Alternative for min coins (shortest path)	Faster for sparse coin sets with large target
Dijkstra	Weighted shortest path	Overkill for coin change (all coins are weight 1)
Subset Sum	0/1 version of coin change	Use when each coin can be used at most once

17. Practice Problems

Easy

1. **Coin Change (Minimum Coins)** — LeetCode 322 — Standard — Medium
2. **Perfect Squares** — LeetCode 279 — Min perfect squares — Medium

Medium

1. **Coin Change II** — LeetCode 518 — Number of combinations — Medium
2. **Combination Sum IV** — LeetCode 377 — Number of permutations — Medium
3. **Minimum Cost For Tickets** — LeetCode 983 — Calendar-based min cost — Medium

Hard

1. **Shopping Offers** — LeetCode 638 — Complex unbounded with bundles — Hard
2. **Minimum Number of Refueling Stops** — LeetCode 871 — Coin-change-like minimisation — Hard

18. Interview Explanation

"Coin Change has two main variants. For **minimum coins**, I use 1D DP where $dp[a]$ = minimum coins to make amount a . I initialise $dp[0] = 0$ and all others to a large sentinel. For each coin, I iterate amounts ascending from coin to target, updating $dp[a] = \min(dp[a], dp[a-coin] + 1)$. The ascending order is critical — it allows unlimited reuse of each coin, making this an unbounded knapsack problem. Time is $O(n \times \text{amount})$, space $O(\text{amount})$. If the target is unreachable, I return -1. For the **number of ways** variant, the recurrence is $dp[a] += dp[a-coin]$ with $dp[0] = 1$. The outer loop must be over coins (not amounts) to count combinations rather than permutations."

19. Revision Notes

- **Min coins:** $dp[a] = \min(dp[a], dp[a-coin] + 1)$, ascending loop.
- **Ways:** $dp[a] += dp[a-coin]$, ascending loop, outer loop = coins (combinations).

- **Base cases:** Min: $dp[0] = 0$, others INF. Ways: $dp[0] = 1$, others 0.
- **Impossible check (min):** `if (dp[amount] == INF) return -1.`
- **Complexity:** $O(n \times \text{amount})$ time, $O(\text{amount})$ space.
- **Traps:** Ascending vs descending; outer loop coin/amount order for ways; overflow in ways.

20. Final Cheat Sheet

COIN CHANGE

When to use: Make amount with unlimited coins – min count (or number of ways)

Recurrence (min): $dp[a] = \min(dp[a], dp[a-\text{coin}] + 1)$ [ascending]

Recurrence (ways): $dp[a] += dp[a-\text{coin}]$ [ascending, coin outer loop]

Base (min): $dp[0] = 0$, others = INF

Base (ways): $dp[0] = 1$, others = 0

Time: $O(n \times \text{amount})$

Space: $O(\text{amount})$

Key code (min):

```
vector<int> dp(amount + 1, amount + 1);
dp[0] = 0;
for (int coin : coins)
    for (int a = coin; a <= amount; ++a)
        dp[a] = min(dp[a], dp[a-coin] + 1);
return dp[amount] == amount+1 ? -1 : dp[amount];
```

Edge cases: amount=0; no coins; unreachable amount

Variations: Perfect squares (same min pattern), combination sum IV (permutations)

12. LONGEST INCREASING SUBSEQUENCE

1. Overview

Given an array of integers, find the length of the **longest strictly increasing subsequence** (LIS). A subsequence is not necessarily contiguous, but elements must be in order. For example, `[10, 9, 2, 5, 3, 7, 101, 18]` has LIS `[2, 5, 7, 101]` of length 4.

2. Intuition

Simple explanation (DP): For each element, the longest increasing subsequence ending at that element is 1 (the element itself) plus the longest such subsequence ending at any previous smaller element.

Better explanation (Patience Sorting): Imagine sorting a deck of cards by placing each card on the leftmost pile whose top card is $\geq$ the card (for increasing). The number of piles at the end is the LIS length. This is a greedy + binary search approach that runs in $O(n \log n)$.

Analogy (Patience Sorting): You're arranging cards into piles. Each pile is decreasing from bottom to top. A new card goes on the leftmost pile where its top card is $\geq$ the new card. If no such pile exists, start a new pile. The number of piles equals the LIS length.

Step-by-step reasoning (DP):

1. Start $dp[i] = 1$ for all i (each element alone is a subsequence of length 1).

- For each index i , check all previous indices $j < i$.
- If $nums[j] < nums[i]$, then $dp[i] = \max(dp[i], dp[j] + 1)$.
- Answer = max over all $dp[i]$.

Why it works (DP): The optimal substructure is: the LIS ending at i is built by extending the LIS ending at some earlier j where $nums[j] < nums[i]$. This is classic DP with $O(n^2)$ complexity.

Why it works (Binary Search): The `tails` array stores the smallest possible tail value for each length of increasing subsequence. For each element, we binary search to find where it fits in `tails`. This maintains optimal tails greedily.

3. When to Use It

- Finding the length of the longest increasing subsequence.
- Any problem about "longest" with an ordering constraint.
- Problems reducible to LIS via transformation (e.g., Russian Doll Envelopes).

Trigger phrases: "Longest increasing subsequence", "LIS", "maximum length subsequence", "non-decreasing", "strictly increasing", "sequence where order matters".

4. When Not to Use It

- When elements can be **reordered** (sorting) — then it's just counting elements in increasing order.
- When the subsequence must be **contiguous** — use sliding window, not LIS.
- When you need the **subsequence itself** (not just length) — both DP and binary search can reconstruct, but DP is easier for reconstruction.
- When the array is **extremely large** ($n > 10^5$) and you need $O(n \log n)$ — use binary search approach.
- When the constraint is **non-decreasing** (allow equal) — change $<$ to $\leq$ in comparisons.

5. Core Concepts

Concept	Explanation	Why It Matters
DP state	$dp[i]$ = LIS length ending at index i	Core DP formulation
Transition (DP)	$dp[i] = \max(dp[i], dp[j] + 1)$ for $j < i$ and $nums[j] < nums[i]$	Extend subsequence
tails array	$tails[l]$ = smallest tail of an increasing subsequence of length l	Key data structure for $O(n \log n)$
Binary search	Find position of $nums[i]$ in <code>tails</code>	Makes the algorithm $O(n \log n)$
Patience sorting	Card-pile analogy for LIS	Intuitive understanding of <code>tails</code>

6. Step-by-Step Algorithm

$O(n^2)$ DP:

- Let $n = nums.size()$.
- Create `dp` of size n , all = 1.
- For $i = 0$ to $n-1$:
 - For $j = 0$ to $i-1$:
 - If $nums[j] < nums[i]$: $dp[i] = \max(dp[i], dp[j] + 1)$.
- Return $\max(dp)$.

O(n log n) Binary Search:

1. Create empty array `tails`.
2. For each `num` in `nums`:
 - Find the leftmost `tails[pos]` where `tails[pos] >= num` using binary search.
 - If found, replace `tails[pos] = num`.
 - If not found (`num >` all tails), append `num` to `tails`.
3. Return `tails.size()`.

7. Dry Run

Input: `nums = [10, 9, 2, 5, 3, 7, 101, 18]`

O(n²) DP:

Index	Value	dp[i]	Extended from
0	10	1	-
1	9	1	-
2	2	1	-
3	5	2	index 2 (2 < 5)
4	3	2	index 2 (2 < 3)
5	7	3	index 3 or 4 (5 < 7 or 3 < 7)
6	101	4	index 5 (7 < 101)
7	18	4	index 5 (7 < 18)

Answer: 4

O(n log n) with `tails`:

num	tails before	binary search	action	tails after
10	[]	-	append	[10]
9	[10]	pos=0 (10≥9)	replace	[9]
2	[9]	pos=0 (9≥2)	replace	[2]
5	[2]	pos=1 (not found)	append	[2, 5]
3	[2, 5]	pos=1 (5≥3)	replace	[2, 3]
7	[2, 3]	pos=2 (not found)	append	[2, 3, 7]
101	[2, 3, 7]	pos=3 (not found)	append	[2, 3, 7, 101]
18	[2, 3, 7, 101]	pos=3 (101≥18)	replace	[2, 3, 7, 18]

Answer: `tails.size() = 4`

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// ----- LIS: O(n^2) DP -----
int lengthOfLIS_DP(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;

    vector<int> dp(n, 1);
    int maxLen = 1;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        maxLen = max(maxLen, dp[i]);
    }
    return maxLen;
}

// ----- LIS: O(n log n) Binary Search -----
// LeetCode 300
int lengthOfLIS(vector<int>& nums) {
    vector<int> tails; // tails[l] = smallest tail for subsequence of length l

    for (int num : nums) {
        // Find first tails[pos] >= num
        auto it = lower_bound(tails.begin(), tails.end(), num);
        if (it == tails.end()) {
            tails.push_back(num); // num > all tails, extend LIS
        } else {
            *it = num; // replace with smaller tail
        }
    }
    return tails.size();
}

// ----- LIS: O(n log n) with custom binary search -----
int lengthOfLIS_ManualBS(vector<int>& nums) {
    vector<int> tails;

    for (int num : nums) {
        int lo = 0, hi = tails.size();
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (tails[mid] < num)
                lo = mid + 1;
            else
                hi = mid;
        }
        // lo is the position to insert/replace
        if (lo == tails.size())
            tails.push_back(num);
        else
            tails[lo] = num;
    }
}

```

```

    }
    return tails.size();
}

// ----- LIS: Reconstruct the actual subsequence -----
vector<int> reconstructLIS(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return {};

    vector<int> dp(n, 1);
    vector<int> prev(n, -1);
    int maxIdx = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                prev[i] = j;
            }
        }
        if (dp[i] > dp[maxIdx])
            maxIdx = i;
    }

    // Reconstruct
    vector<int> lis;
    for (int i = maxIdx; i != -1; i = prev[i])
        lis.push_back(nums[i]);
    reverse(lis.begin(), lis.end());
    return lis;
}

// ----- Example usage -----
int main() {
    vector<int> nums = {10, 9, 2, 5, 3, 7, 101, 18};

    cout << "LIS length (DP): " << lengthOfLIS_DP(nums) << "\n";           // 4
    cout << "LIS length (BS): " << lengthOfLIS(nums) << "\n";             // 4
    cout << "LIS length (Manual BS): " << lengthOfLIS_ManualBS(nums) << "\n"; // 4

    auto lis = reconstructLIS(nums);
    cout << "LIS: ";
    for (int x : lis) cout << x << " ";
    cout << "\n"; // 2 5 7 101 or 2 3 7 101

    return 0;
}

```

9. Python Implementation

```

import bisect

# ----- LIS: O(n^2) DP -----
def length_of_lis_dp(nums: list[int]) -> int:
    if not nums:
        return 0

```

```

dp = [1] * len(nums)
for i in range(len(nums)):
    for j in range(i):
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)
return max(dp)

# ----- LIS: O(n log n) Binary Search -----
def length_of_lis(nums: list[int]) -> int:
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)

# ----- LIS: Reconstruct -----
def reconstruct_lis(nums: list[int]) -> list[int]:
    if not nums:
        return []

    n = len(nums)
    dp = [1] * n
    prev = [-1] * n
    max_idx = 0

    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i] and dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
                prev[i] = j
        if dp[i] > dp[max_idx]:
            max_idx = i

    lis = []
    i = max_idx
    while i != -1:
        lis.append(nums[i])
        i = prev[i]
    lis.reverse()
    return lis

# Example
nums = [10, 9, 2, 5, 3, 7, 101, 18]
print(length_of_lis(nums)) # 4
print(reconstruct_lis(nums)) # [2, 5, 7, 101]

```

10. Code Explanation

- **$O(n^2)$ DP:** Simple and intuitive. $dp[i]$ = LIS ending at index i . Check all previous $j < i$: if $nums[j] < nums[i]$, we can extend the subsequence ending at j . Answer = $\max(dp)$.
- **$O(n \log n)$ with `tails`:** `tails[l]` stores the smallest possible tail value for an increasing subsequence of length l . Initially empty. For each `num`, `lower_bound` finds the first position where `tails[pos] >= num`. If found, replace

with `num` (smaller tail makes it easier to extend later). If not found (`num > all tails`), append — this increases the LIS length. The result is `tails.size()`.

- **lower_bound vs upper_bound:** For **strictly increasing**, use `lower_bound` (first $\geq$). For **non-decreasing**, use `upper_bound` (first $>$).
- **Reconstruction:** `prev[i]` stores the index of the previous element in the LIS ending at `i`. We track `maxIdx` — the index where the longest LIS ends. Then backtrack using `prev` and reverse.

11. Complexity Analysis

Variant	Time	Space	Notes
$O(n^2)$ DP	$O(n^2)$	$O(n)$	Simple, works for $n \leq 2000$
$O(n \log n)$ BS	$O(n \log n)$	$O(n)$	Standard for $n \leq 10^5$
Reconstruction (DP)	$O(n^2)$	$O(n)$	Add prev array
Reconstruction (BS)	$O(n \log n)$	$O(n)$	More complex but possible

12. Common Patterns

Pattern	Identification	Approach	Example
Standard LIS	"Longest increasing subsequence"	Binary search / DP	LeetCode 300
Non-decreasing	"Allow equal elements"	Use <code>upper_bound</code> instead of <code>lower_bound</code>	Variation
Russian Doll Envelopes	"Sort by width, LIS on height"	Sort + LIS	LeetCode 354
Maximum Sum Increasing Subsequence	"Max sum, not max length"	DP with sum instead of count	GFG
Number of LIS	"Count subsequences of max length"	DP with count array	LeetCode 673
Longest Chain of Pairs	"Pair (a,b) must have $a < b$ and chain connects"	Sort + LIS	LeetCode 646

13. Common Mistakes

- **Using `lower_bound` when non-decreasing is needed:** For non-decreasing (allow equal), use `upper_bound`.
- **Confusing `tails` with actual LIS:** `tails` is NOT the LIS subsequence — it's an array of smallest possible tails. It only gives the correct length, not the actual sequence.
- **Forgetting that `tails` must be initialised empty:** Starting with a large number in tails causes wrong results.
- **$O(n^2)$ overflow for $n > 10^4$:** Use binary search approach.
- **Not handling empty array:** Return 0.
- **Reconstruction from `tails`:** Not directly possible — reconstruction needs the 2D DP or parent pointers.

14. Edge Cases

Case	Input	Expected	Reason
Empty array	<code>[]</code>	0	No elements

Case	Input	Expected	Reason
Single element	[5]	1	Just that element
All decreasing	[5, 4, 3, 2, 1]	1	Any single element
All increasing	[1, 2, 3, 4, 5]	5	Whole array
All equal	[5, 5, 5, 5]	1 (strict) or 4 (non-decreasing)	Strict = 1
Duplicates	[1, 3, 3, 4]	3 (1,3,4)	Strict excludes equal
Large n	10 ⁵ elements	O(n log n) needed	Binary search method

15. Variations

Variation	Description	Importance
Non-decreasing LIS	Allow equal elements	Use <code>upper_bound</code>
Maximum Sum Increasing Subsequence	Max sum instead of max length	Medium — GFG
Number of LIS	Count how many LIS of max length	Medium — LeetCode 673
Russian Doll Envelopes	2D: sort by one dim, LIS on other	Hard — LeetCode 354
Longest Chain of Pairs	Sort by first element, LIS on second	Medium — LeetCode 646
Longest Bitonic Subsequence	Increasing then decreasing	Medium — GFG

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Patience Sorting	Same as binary search method	Card pile analogy
Longest Common Subsequence (LCS)	Another classic subsequence DP	LCS doesn't require order preservation beyond original sequence
Maximum Subarray (Kadane)	Contiguous vs non-contiguous	Kadane for contiguous, LIS for not necessarily contiguous
Edit Distance	Both are subsequence-based DPs	Different objectives (min edits vs max length)

17. Practice Problems

Easy

1. **Longest Increasing Subsequence** — LeetCode 300 — Standard — Medium
2. **Longest Increasing Subsequence (GFG)** — Standard — Medium

Medium

1. **Number of Longest Increasing Subsequence** — LeetCode 673 — Count LIS — Medium
2. **Maximum Length of Pair Chain** — LeetCode 646 — Chain pairs — Medium
3. **Longest Increasing Subsequence (CSES)** — Standard — Medium

Hard

1. **Russian Doll Envelopes** — LeetCode 354 — 2D LIS — Hard
2. **Minimum Number of Removals to Make Mountain Array** — LeetCode 1671 — Bitonic LIS — Hard

18. Interview Explanation

"For LIS, I'd use the $O(n \log n)$ binary search approach. I maintain an array `tails` where `tails[1]` stores the smallest possible tail value for an increasing subsequence of length 1. For each element, I use binary search (`lower_bound`) to find the first tail $\geq$ the current element. If found, I replace it with the smaller value — this keeps tails optimal without changing the length. If the element is larger than all tails, I append it, increasing the LIS length by 1. The answer is the size of `tails`. This works because replacing a tail with a smaller value never hurts — it can only make it easier to extend subsequences later. The time complexity is $O(n \log n)$ and space $O(n)$. I'd also mention the $O(n^2)$ DP approach as a simpler alternative when n is small."

19. Revision Notes

- **Key idea (BS):** `tails` array stores smallest possible tail for each LIS length.
- **Key idea (DP):** `dp[i] = max(dp[j] + 1)` for $j < i$ and `nums[j] < nums[i]`.
- **Lower bound:** `lower_bound` for strict increasing; `upper_bound` for non-decreasing.
- **Formula (BS):** `tails[pos] = num` where `pos = lower_bound(tails, num)`.
- **Complexity:** $O(n^2)$ DP, $O(n \log n)$ BS.
- **Traps:** `tails` is NOT the actual LIS; strict vs non-decreasing distinction; empty array.

20. Final Cheat Sheet

LONGEST INCREASING SUBSEQUENCE

When to use: Longest subsequence where elements increase (not necessarily contiguous)

Recurrence (DP): `dp[i] = max(dp[j] + 1)` for $j < i$, `nums[j] < nums[i]`

Method (BS): `tails[pos] = num`, `pos = lower_bound(tails, num)`
 if `pos == tails.size()`: append; else: replace

Time: $O(n^2)$ DP, $O(n \log n)$ BS

Space: $O(n)$

Key code (BS):

```
vector<int> tails;
for (int num : nums) {
    auto it = lower_bound(tails.begin(), tails.end(), num);
    if (it == tails.end()) tails.push_back(num);
    else *it = num;
}
return tails.size();
```

Edge cases: Empty $\rightarrow 0$; all decreasing $\rightarrow 1$; all equal $\rightarrow 1$ (strict)

Variations: Non-decreasing, max sum LIS, number of LIS, Russian dolls

13. LONGEST COMMON SUBSEQUENCE

1. Overview

Given two strings (or sequences) `text1` and `text2`, find the length of their **longest common subsequence** (LCS). A subsequence is a sequence that appears in the same relative order but not necessarily contiguously. For example, LCS of "abcde" and "ace" is "ace" of length 3.

2. Intuition

Simple explanation: We compare characters from both strings one by one. If characters match, we can extend the common subsequence by 1. If they don't match, we take the best we can do by skipping a character from either string.

Analogy: You have two sequences of moves in a dance. You want to find the longest sequence of moves that appear in both dances in the same order, even if there are extra moves in between. At each step, either the moves match (use it) or they don't (skip one move from either dance and check the rest).

Step-by-step reasoning:

1. Let `m = len(text1)`, `n = len(text2)`.
2. Create DP table `dp[m+1][n+1]`, all 0.
3. For `i = 1` to `m`, for `j = 1` to `n`:
 - If `text1[i-1] == text2[j-1]`: `dp[i][j] = dp[i-1][j-1] + 1`.
 - Else: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.
4. Answer = `dp[m][n]`.

Why it works: `dp[i][j]` represents LCS of prefixes `text1[0..i-1]` and `text2[0..j-1]`. When characters match, we take the LCS of the prefixes before these characters and add 1. When they don't match, we carry forward the best LCS from either skipping the current character of `text1` or `text2`. This is optimal substructure: the LCS of two prefixes is built from LCS of smaller prefixes.

3. When to Use It

- Finding the longest common subsequence between two strings.
- Problems involving "similarity" or "matching" between two sequences.
- As a building block for edit distance (adding/deleting characters).
- Bioinformatics (DNA sequence alignment).

Trigger phrases: "Longest common subsequence", "LCS", "common in both sequences", "subsequence matching", "edit distance related".

4. When Not to Use It

- When you need **contiguous** subsequences (substrings) — use longest common substring (different DP).
- When you need the **shortest common supersequence** — use LCS result but different construction.
- When strings are **very long** (10^5) — $O(mn)$ may be too slow; use Hirschberg's algorithm or suffix automaton for special cases.
- When you need **all** common subsequences (not just longest) — combinatorial explosion.
- When the strings contain **only one character type** — trivial, answer is $\min(\text{len1}, \text{len2})$.

5. Core Concepts

Concept	Explanation	Why It Matters
State	<code>dp[i][j]</code> = LCS of <code>text1[0..i-1]</code> and <code>text2[0..j-1]</code>	2D DP table
Match case	<code>dp[i][j] = dp[i-1][j-1] + 1</code>	Extend common subsequence

Concept	Explanation	Why It Matters
Mismatch case	$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$	Carry forward best
Space optimisation	Only need 2 rows (current and previous)	Reduces memory from $O(mn)$ to $O(n)$
Backtracking	Trace through DP table to reconstruct LCS	Get the actual subsequence
Non-contiguous nature	LCS $\neq$ substring	Why we don't reset on mismatch

6. Step-by-Step Algorithm

- Let $m = \text{len}(\text{text1})$, $n = \text{len}(\text{text2})$.
- Create dp of size $(m+1) \times (n+1)$, initialised to 0.
- For $i = 1$ to m :
 - For $j = 1$ to n :
 - If $\text{text1}[i-1] == \text{text2}[j-1]$:
 - $dp[i][j] = dp[i-1][j-1] + 1$.
 - Else:
 - $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Return $dp[m][n]$.

7. Dry Run

Input: $\text{text1} = \text{"abcde"}$, $\text{text2} = \text{"ace"}$

DP Table:

	$\emptyset$	a	c	e
$\emptyset$	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

Step-by-step:

- $i=1$ (a) vs $j=1$ (a): match $\rightarrow dp[1][1] = dp[0][0] + 1 = 1$.
- $i=1$ (a) vs $j=2$ (c): no match $\rightarrow dp[1][2] = \max(dp[0][2]=0, dp[1][1]=1) = 1$.
- $i=1$ (a) vs $j=3$ (e): no match $\rightarrow dp[1][3] = \max(dp[0][3]=0, dp[1][2]=1) = 1$.
- $i=2$ (b) vs $j=1$ (a): no match $\rightarrow dp[2][1] = \max(dp[1][1]=1, dp[2][0]=0) = 1$.
- ... continuing the same pattern.
- $i=3$ (c) vs $j=2$ (c): match $\rightarrow dp[3][2] = dp[2][1] + 1 = 1 + 1 = 2$.
- $i=5$ (e) vs $j=3$ (e): match $\rightarrow dp[5][3] = dp[4][2] + 1 = 2 + 1 = 3$.

Answer: 3 ("ace")

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// ----- LCS: Standard 2D DP -----
// LeetCode 1143
int longestCommonSubsequence(string text1, string text2) {
    int m = text1.size(), n = text2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}

// ----- LCS: Space Optimised (2 rows) -----
int longestCommonSubsequenceOptimised(string text1, string text2) {
    int m = text1.size(), n = text2.size();
    vector<int> prev(n + 1, 0), curr(n + 1, 0);

    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                curr[j] = prev[j - 1] + 1;
            } else {
                curr[j] = max(prev[j], curr[j - 1]);
            }
        }
        swap(prev, curr);
    }
    return prev[n];
}

// ----- LCS: Reconstruct the actual subsequence -----
string reconstructLCS(string text1, string text2) {
    int m = text1.size(), n = text2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; ++i)
        for (int j = 1; j <= n; ++j)
            if (text1[i - 1] == text2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);

    // Backtrack
    string lcs;
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (text1[i - 1] == text2[j - 1]) {
            lcs.push_back(text1[i - 1]);

```

```

        --i; --j;
    } else if (dp[i - 1][j] > dp[i][j - 1]) {
        --i; // Move up
    } else {
        --j; // Move left
    }
}
reverse(lcs.begin(), lcs.end());
return lcs;
}

// ----- Example usage -----
int main() {
    string text1 = "abcde", text2 = "ace";

    cout << "LCS length: " << longestCommonSubsequence(text1, text2) << "\n"; // 3
    cout << "LCS string: \"\" << reconstructLCS(text1, text2) << "\"\n";      // "ace"

    // Space optimised
    cout << "LCS length (optimised): "
        << longestCommonSubsequenceOptimised(text1, text2) << "\n"; // 3

    return 0;
}

```

9. Python Implementation

```

# ----- LCS: Standard 2D DP -----
def longest_common_subsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[m][n]

# ----- LCS: Space Optimised (2 rows) -----
def lcs_optimised(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    prev = [0] * (n + 1)
    curr = [0] * (n + 1)

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                curr[j] = prev[j - 1] + 1
            else:
                curr[j] = max(prev[j], curr[j - 1])
        prev, curr = curr, prev # Swap references

    return prev[n]

```

```
# ----- LCS: Reconstruct -----
def reconstruct_lcs(text1: str, text2: str) -> str:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    # Backtrack
    lcs = []
    i, j = m, n
    while i > 0 and j > 0:
        if text1[i - 1] == text2[j - 1]:
            lcs.append(text1[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] > dp[i][j - 1]:
            i -= 1
        else:
            j -= 1

    return ''.join(reversed(lcs))

# Example
text1, text2 = "abcde", "ace"
print(longest_common_subsequence(text1, text2)) # 3
print(reconstruct_lcs(text1, text2)) # "ace"
```

10. Code Explanation

- **2D DP table:** `dp[i][j]` stores LCS of `text1[0..i-1]` and `text2[0..j-1]`. Size is $(m+1) \times (n+1)$ to include empty string prefix.
- **Match case:** `dp[i][j] = dp[i-1][j-1] + 1` — we found a common character, so extend the LCS of the prefixes before these characters.
- **Mismatch case:** `dp[i][j] = max(dp[i-1][j], dp[i][j-1])` — we skip the character from either text1 (move up) or text2 (move left), taking the better of the two.
- **Space optimisation:** Only two rows (`prev` for `i-1`, `curr` for `i`) are needed because `dp[i][j]` only references `dp[i-1][j-1]`, `dp[i-1][j]`, and `dp[i][j-1]`. The `swap(prev, curr)` at the end of each row resets for the next iteration.
- **Backtracking:** Starting from `dp[m][n]`, if characters match, we take the character and move diagonally. If they don't match, we move in the direction of the larger value (up or left). The characters collected in reverse order form the LCS.
- **Empty prefix row/column:** `dp[0][j] = 0` and `dp[i][0] = 0` are the base cases — LCS with an empty string is always 0.

11. Complexity Analysis

Variant	Time	Space	Notes
2D DP	$O(m \times n)$	$O(m \times n)$	Standard, good for interviews
2-row (space optimised)	$O(m \times n)$	$O(n)$	For CP with memory constraints
1-row (further opt)	$O(m \times n)$	$O(n)$	Same as 2-row in practice
With reconstruction	$O(m \times n)$	$O(m \times n)$	Need full table for backtracking

12. Common Patterns

Pattern	Identification	Approach	Example
Standard LCS	"Longest common subsequence of two strings"	$dp[i][j] = \text{match?} + 1 : \max(\text{top}, \text{left})$	LeetCode 1143
Shortest Common Supersequence	"Shortest string containing both as subsequences"	$m + n - \text{LCS length}$	LeetCode 1092
Edit Distance	"Min edits to transform one string to another"	Add replace/insert/delete costs	LeetCode 72
Longest Palindromic Subsequence	"LPS of a string"	LCS of string with its reverse	LeetCode 516
Minimum Insertions for Palindrome	"Min insertions to make palindrome"	$n - \text{LPS length}$	LeetCode 1312

13. Common Mistakes

- **Off-by-one with indices:** `text1[i-1]` vs `text2[j-1]` — the dp array is 1-indexed but the string is 0-indexed.
- **Confusing with longest common substring:** Substring DP resets on mismatch ($dp[i][j] = 0$), subsequence takes `max`.
- **Not handling empty strings:** The base cases $dp[0][j] = 0$ and $dp[i][0] = 0$ handle this automatically.
- **Reconstruction with wrong backtracking logic:** Always move towards the larger value. If equal, either direction works but may give different valid LCS.
- **Space optimisation row swap order:** Must `swap(prev, curr)` after each row, or manually reset `curr` and use `prev` for the previous row.
- **Forgetting to reverse reconstructed LCS:** Backtracking goes from end to start, so characters must be reversed.

14. Edge Cases

Case	Input	Expected	Reason
Both empty	<code>"", ""</code>	0	No common characters
One empty	<code>"abc", ""</code>	0	Empty subsequence
No common	<code>"abc", "def"</code>	0	No matching characters
Identical	<code>"abc", "abc"</code>	3	Whole string
One char common	<code>"abc", "xyzc"</code>	1	Just 'c'

Case	Input	Expected	Reason
Reversed	"abc", "cba"	1	Only one in common at same position? Actually LCS of "abc" and "cba" is 1 (a, b, or c)
Large strings	1000 × 1000	Fits O(mn)	Need ~4MB for int DP

15. Variations

Variation	Description	Importance
Shortest Common Supersequence	Shortest string containing both	Medium — LeetCode 1092
Edit Distance	Min edits (insert/delete/replace)	High — LeetCode 72
Longest Palindromic Subsequence	LPS of one string	High — LeetCode 516
Minimum Insertions to Make Palindrome	n - LPS length	Medium — LeetCode 1312
Longest Common Substring	Contiguous version	Medium — different DP
LCS of 3 strings	LCS across three sequences	Hard — harder DP

16. Related Algorithms/Data Structures

Algorithm	Connection	How to Choose
Edit Distance	LCS is a special case (match/mismatch only)	Use edit distance for weighted operations
Longest Common Substring	Contiguous version of LCS	Reset dp to 0 on mismatch for substring
Longest Palindromic Subsequence	LCS(string, reverse(string))	Use LCS directly
Rabin-Karp / Rolling Hash	Alternative for substring matching	Not directly related to LCS

17. Practice Problems

Easy

1. **Longest Common Subsequence** — LeetCode 1143 — Standard — Medium
2. **Longest Common Subsequence (GFG)** — Standard — Medium

Medium

1. **Shortest Common Supersequence** — LeetCode 1092 — Based on LCS length — Hard
2. **Longest Palindromic Subsequence** — LeetCode 516 — LCS with reverse — Medium
3. **Edit Distance** — LeetCode 72 — Generalised LCS — Medium

Hard

1. **Minimum Insertion Steps to Make a String Palindrome** — LeetCode 1312 — Based on LPS — Hard
2. **Distinct Subsequences** — LeetCode 115 — Counting subsequences — Hard

18. Interview Explanation

"LCS is a classic 2D DP problem. I build a table where $dp[i][j]$ represents the LCS of the first i characters of `text1` and the first j characters of `text2`. If the current characters match, we add 1 to the LCS of the prefixes without them — $dp[i-1][j-1] + 1$. If they don't match, we take the maximum of skipping a character from either string — $\max(dp[i-1][j], dp[i][j-1])$. The base cases are $dp[0][j] = 0$ and $dp[i][0] = 0$ since LCS with an empty string is empty. Time is $O(mn)$ and space can be optimised to $O(n)$ using only two rows since we only need the previous row. For reconstruction, I backtrack through the table with parent pointers."

19. Revision Notes

- **Key idea:** Match → extend; mismatch → carry forward max.
- **Formula:** `if (s1[i]==s2[j]): dp[i][j] = dp[i-1][j-1] + 1; else: dp[i][j] = max(dp[i-1][j], dp[i][j-1]).`
- **Base cases:** `dp[0][j] = dp[i][0] = 0.`
- **Complexity:** $O(mn)$ time, $O(mn)$ or $O(n)$ space.
- **Traps:** Off-by-one with string indices; confusing substring vs subsequence; reconstruction direction.
- **Space optimisation:** Two rows, `prev` and `curr`, swap after each row.

20. Final Cheat Sheet

LONGEST COMMON SUBSEQUENCE

When to use: Find common subsequence (not contiguous) of two strings

Recurrence: `if s1[i]==s2[j]: dp[i][j] = dp[i-1][j-1] + 1`
`else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

Base cases: `dp[0][*] = dp[*][0] = 0`

Time: $O(m \times n)$

Space: $O(m \times n)$ or $O(n)$ with rolling rows

Key code:

```
vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
for (int i = 1; i <= m; ++i)
    for (int j = 1; j <= n; ++j)
        if (text1[i-1] == text2[j-1])
            dp[i][j] = dp[i-1][j-1] + 1;
        else
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
return dp[m][n];
```

Edge cases: Empty strings, no match, identical strings

Variations: Shortest common supersequence, edit distance, LPS

End of Common DP Patterns — Part 1

This document covers 14 essential DP patterns for placements and competitive programming. Practice consistently, drill the templates, and recognise patterns in new problems. Good luck with your preparation!